

Logistic Regression, Classifier Accuracy

Assigned reading:

- 5.4-5.4.4 Discriminative Classifiers, Logistic Regression
- 5.2.5, 5.2.6 Classifier Accuracy, ROC

CS 189/289

Today's lecture outline:

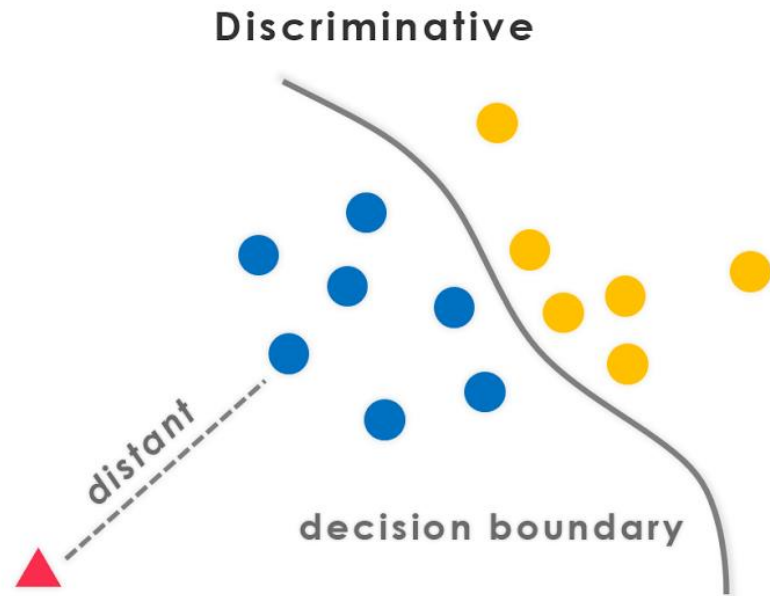
1. Logistic Regression
2. Evaluating Classifiers

CS 189/289

Today's lecture outline:

1. Logistic Regression
2. Evaluating Classifiers

Recall from last class

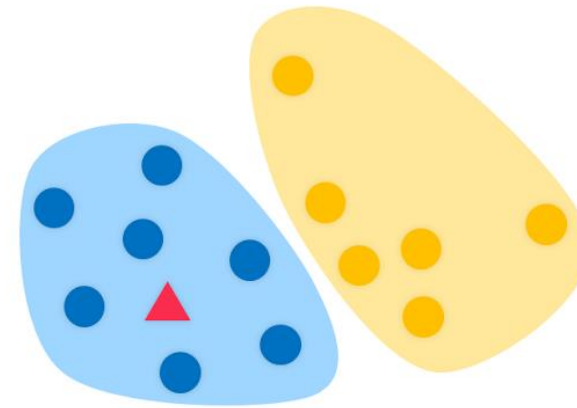


$$p(y|X)$$

e.g., $p(y = C|X, w) = \text{sigmoid}(w^T x)$

Logistic regression

Generative



$$p(y|X) \propto p(X, y) = p(X|y)p(y)$$

e.g., $p(X|y = C, \mu_c, \Sigma_c) = N(X|\mu_c, \Sigma_c)$

$$p(y = 1) = 0.22$$

$$p(y = 0) = 0.78$$

class-conditional gaussians

Recall from last class: Predictive distribution for LDA

- Desired predictive conditional is given by:

$$\begin{aligned} p(Y = a|X) &\propto p(X|Y = a)p(Y = a) \\ &= N(\mu_a, \Sigma)p(Y = a) \\ &\propto \exp[-(X - \mu_a)^T \Sigma^{-1}(X - \mu_a)] \end{aligned}$$

- Similarly for $p(Y = b|X)$.
- If normalize so that $p(Y = a|X) + p(Y = b|X) = 1$, then

$$p(Y = a|X) = \frac{1}{1 + \exp(-w^T X)}$$

- Training this model directly is the discriminative classifier called *logistic regression*!

Predictive distribution for any generative model

$$\begin{aligned} p(\mathcal{C}_1|\mathbf{x}) &= \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1) + p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)} \\ &= \frac{1}{1 + \exp(-a)} = \sigma(a) \end{aligned}$$

where we have defined

$$a = \ln \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)}$$

and $\sigma(a)$ is the *logistic sigmoid* function defined by

$$\sigma(a) = \frac{1}{1 + \exp(-a)},$$

Recall from last class: Predictive distribution for LDA

$$\begin{aligned}\mathbf{w} &= \Sigma^{-1}(\mu_1 - \mu_2) \\ w_0 &= -\frac{1}{2}\mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2}\mu_2^T \Sigma^{-1} \mu_2 + \ln \frac{p(\mathcal{C}_1)}{p(\mathcal{C}_2)}.\end{aligned}$$

- If normalize so that $p(Y = a|X) + p(Y = b|X) = 1$, then

$$p(Y = a|X) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{X})}$$

- Training this model directly is the discriminative classifier called *logistic regression*!

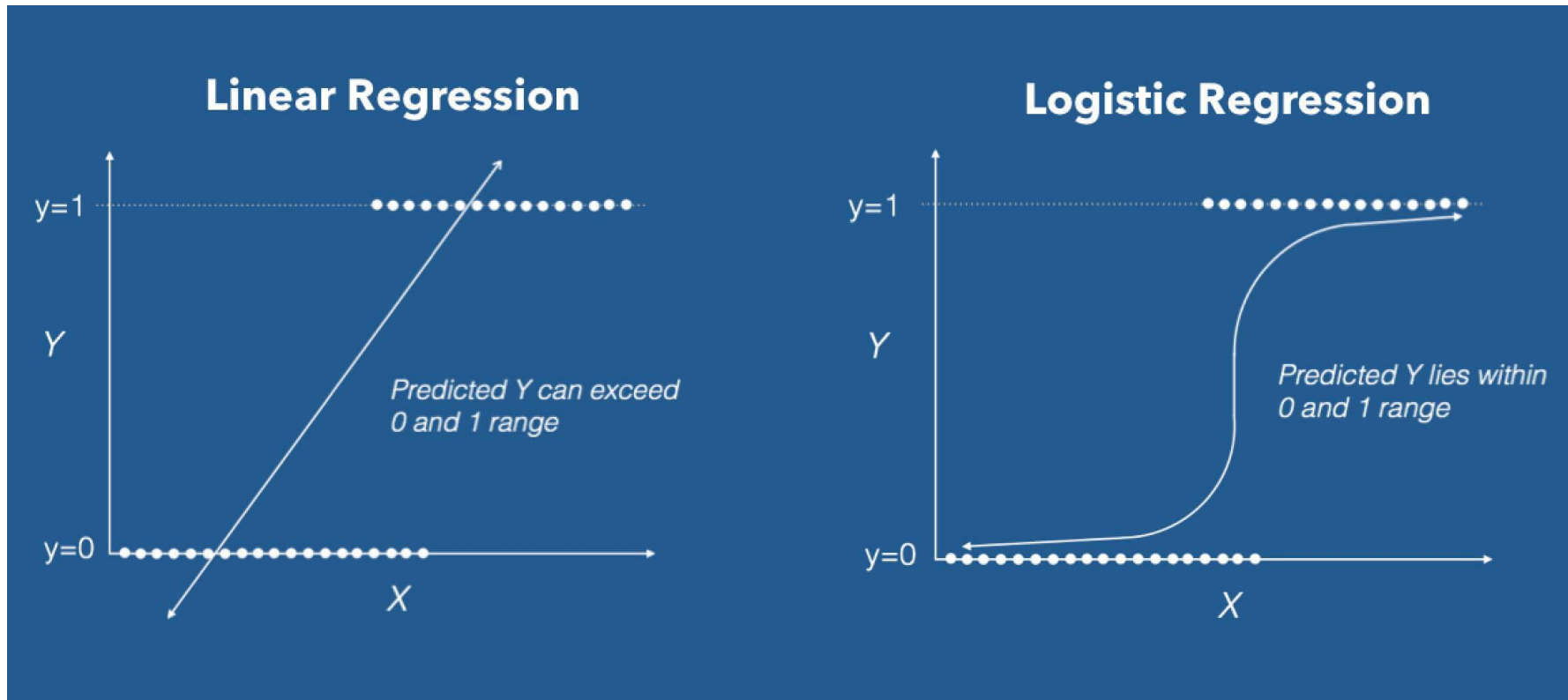
Logistic regression: discriminative classification model

$$p(y_i^{pred} = a | x_i) = \frac{1}{1 + \exp(-w^T x_i)} = \sigma(w^T x_i)$$

1. $y_i^{pred} \in \{a, b\}$
2. For binary classification, classes a and b:
 - $p(y_i = a | x_i, w) = \text{sigmoid}(w^T x_i)$
 - $p(y_i = b | x_i, w) = 1 - p(y_i = a | x_i, w)$
 - $p(y_i = b | x_i, w) + p(y_i = a | x_i, w) = 1$
3. "Hard" class prediction with $\arg \max_k p(y_i^{pred} = k)$

The sigmoid squashes $w^T x_i$ to lie within $[0,1]$

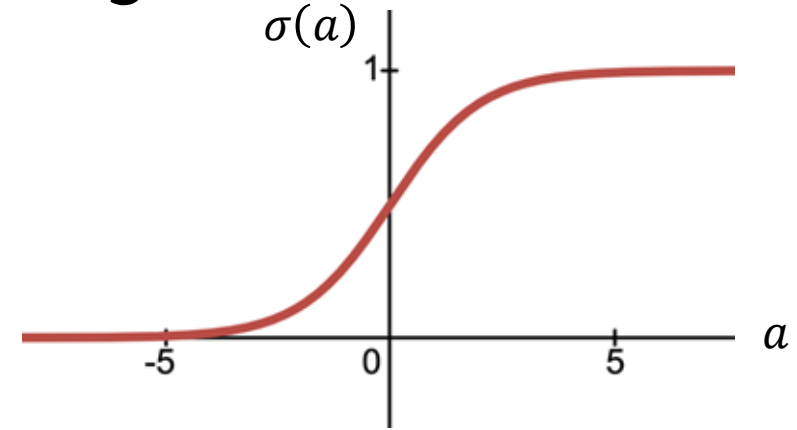
$$p(y_i^{pred} = a | x_i) = \frac{1}{1 + \exp(-w^T x_i)} = \sigma(w^T x_i)$$



The Sigmoid (σ) Function

The S-shaped curve is formally known as the **sigmoid function**.

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$



Reflection/
Symmetry

$$1 - \sigma(a) = \frac{e^{-a}}{1 + e^{-a}} = \sigma(-a)$$

Domain
 $-\infty < a < +\infty$

Range
 $0 < \sigma(a) < 1$

Inverse

$$a = \sigma^{-1}(p) = \log\left(\frac{p}{1-p}\right)$$

Logit Function

Derivative

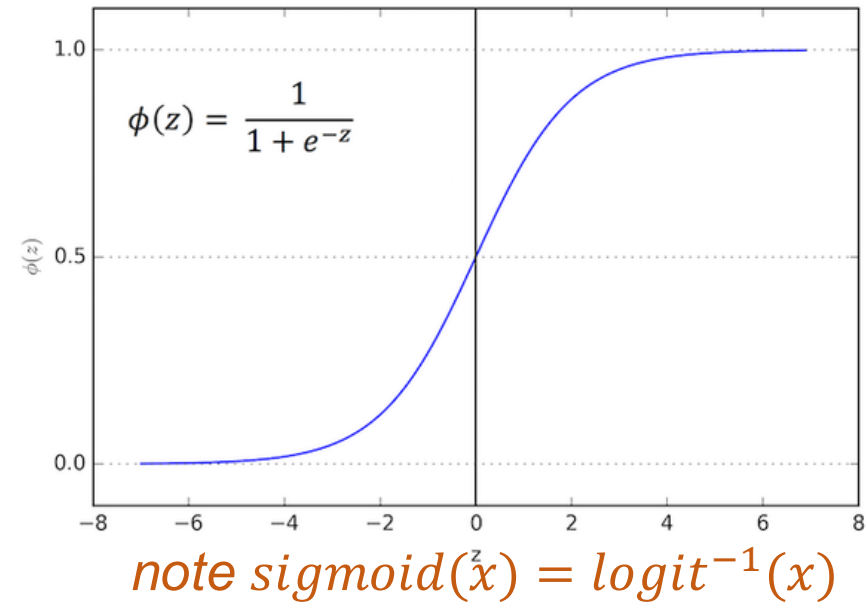
$$\frac{d}{da} \sigma(a) = \sigma(a)(1 - \sigma(a)) = \sigma(a)\sigma(-a)$$

Logistic regression

$$p(y_i^{pred} = a | x_i) = \frac{1}{1 + \exp(-w^T x_i)}$$

Thought experiment:

1. Imagine we double $w^T x_i$ in linear regression. What would happen?

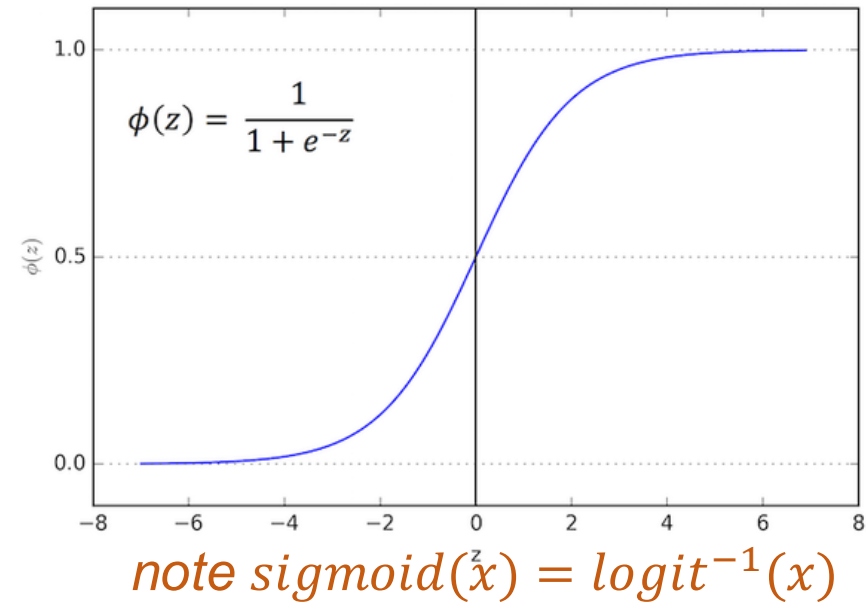


Logistic regression

$$p(y_i^{pred} = a | x_i) = \frac{1}{1 + \exp(-w^T x_i)}$$

Thought experiment:

1. Imagine we double $w^T x_i$ in linear regression. What would happen?
 $\Rightarrow$ *Predictions would be twice as large.*
2. What if we did this again, but **now for logistic regression?**

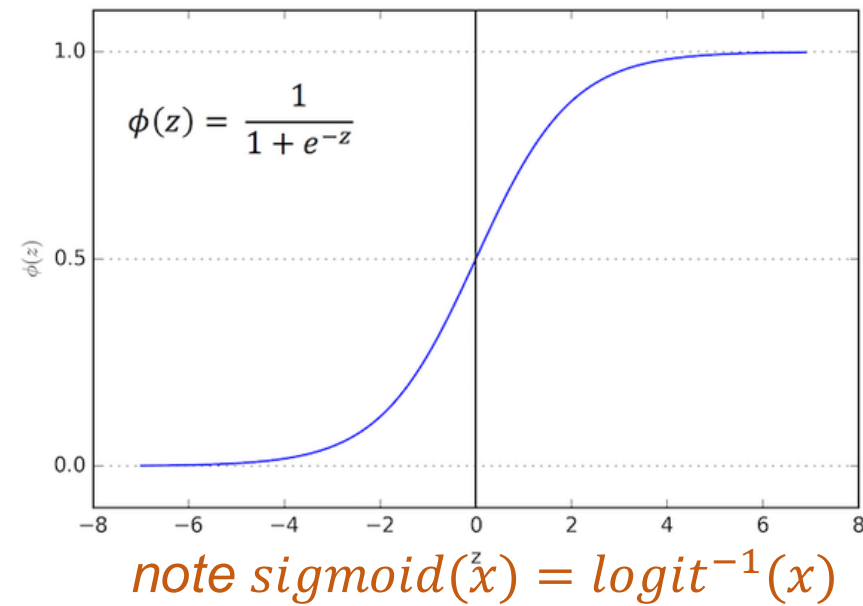


Logistic regression

$$p(y_i^{pred} = a | x_i) = \frac{1}{1 + \exp(-w^T x_i)}$$

Thought experiment:

1. Imagine we double $w^T x_i$ in linear regression. What would happen?
 $\Rightarrow$ Predictions would be twice as large.
2. What if we did this again, but now for logistic regression?
 $\Rightarrow$ Predictions would be more confident.



Multi-class logistic regression

$$\begin{aligned} p(\mathcal{C}_k|\mathbf{x}) &= \frac{p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)}{\sum_j p(\mathbf{x}|\mathcal{C}_j)p(\mathcal{C}_j)} \\ &= \frac{\exp(a_k)}{\sum_j \exp(a_j)}, \end{aligned}$$

$$\begin{aligned} p(\mathcal{C}_1|\mathbf{x}) &= \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1) + p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)} \\ &= \frac{1}{1 + \exp(-a)} = \sigma(a) \end{aligned}$$

"Softmax" is a generalization of sigmoid to $K > 2$.

$$\text{softmax}(j, \{z_1, z_2, \dots, z_K\}) = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

1. Returns values between 0 and 1.

2. Normalized wrt classes: $\mathbf{1} = \sum_{j=1}^K \text{softmax}(j, \{z_1, \dots, z_K\})$

➤ Can choose to treat its outputs as probabilities.

Multi-class logistic regression

$$p(\mathcal{C}_1|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1) + p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)}$$

```
In [52]: data=np.array([-5,-3,-7])

In [53]: softmax(data)
Out[53]: array([ 0.11731043,  0.86681333,  0.01587624])

In [54]: softmax(0.00001*data)
Out[54]: array([ 0.33333333,  0.33334    ,  0.33332667])

In [55]: softmax(100*data)
Out[55]: array([ 1.38389653e-087,  1.00000000e+000,  1.91516960e-174])
```

$K > 2$.

1. Returns values between 0 and 1.
2. Normalized wrt classes: $\mathbf{1} = \sum_{j=1}^K \text{softmax}(j, \{z_1, \dots, z_K\})$

➤ Can choose to treat its outputs as probabilities.

From $K = 2$ ("binary") to $K > 2$ ("multi-class")

Binary logistic regression:

$$\begin{aligned} p(y_i^{\text{pred}} = a | \mathbf{x}_i) &= \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x}_i)} \\ &= \frac{\exp(\mathbf{w}^T \mathbf{x}_i)}{\exp(\mathbf{w}^T \mathbf{x}_i) + 1} = \frac{\exp(\mathbf{w}^T \mathbf{x}_i)}{\exp(\mathbf{w}^T \mathbf{x}_i) + \exp(\mathbf{w}'^T \mathbf{x}_i)} \quad \text{for } \mathbf{w}' = 0 \end{aligned}$$

But softmax is translation invariant:

$$\frac{\exp(z_a + c)}{\exp(z_a + c) + \exp(z_b + c)} = \frac{\exp(z_a) \exp(c)}{\exp(z_a) \exp(c) + \exp(z_b) \exp(c)} = \frac{\exp(z_a)}{\exp(z_a) + \exp(z_b)}$$

- Thus there are infinitely many solutions $(\mathbf{w}, \mathbf{w}')$.
- The system is overparameterized.

From $K = 2$ ("binary") to $K > 2$ ("multi-class")

Binary logistic regression is thus over-parameterized. For any class j ,

$$p(y_i = j | x) = 1 - \sum_{k \neq j} p(y_i = k | x) \quad \text{for } w' = 0$$

Can be re-formulated as multi-class:

$$\begin{aligned} p(y_i^{pred} = a | x_i) &\propto \exp(w_a^T x_i) \\ p(y_i^{pred} = b | x_i) &\propto \exp(w_b^T x_i) \\ \Rightarrow p(y_i^{pred} = a | x_i) &= \frac{\exp(w_a^T x_i)}{\exp(w_a^T x_i) + \exp(w_b^T x_i)} \\ &= \text{softmax}(1, \{w_a^T x_i, w_b^T x_i\}) \end{aligned}$$

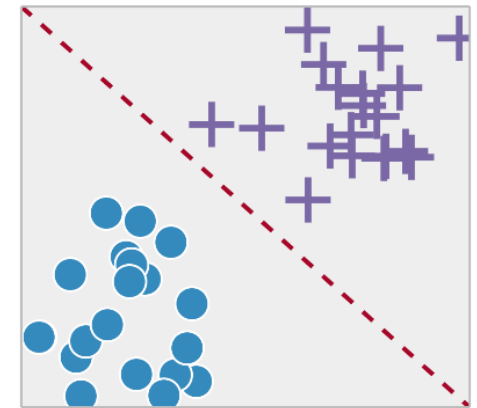
So are these the same model?!

Logistic regression decision boundary

$$p(y_i^{\text{pred}} = a | \mathbf{x}) = \frac{\exp(w_a^T \mathbf{x}_i)}{\exp(w_b^T \mathbf{x}_i) + \exp(w_a^T \mathbf{x}_i)}$$

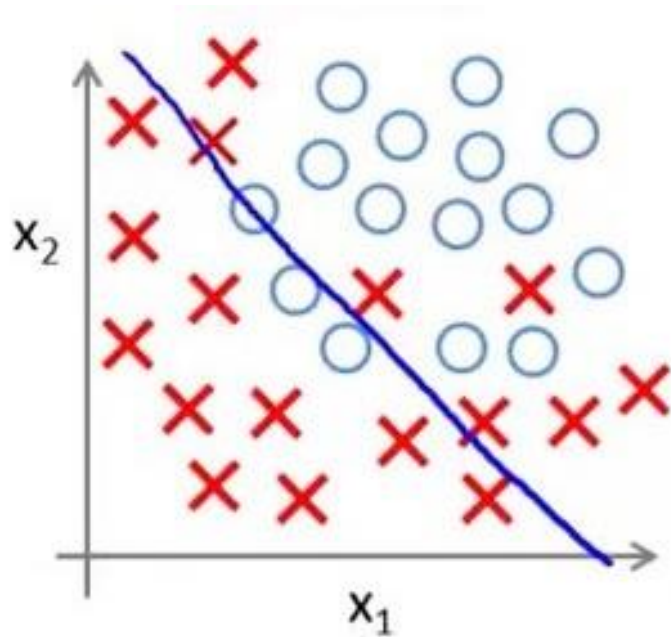
Then decision boundary is:

$$\begin{aligned} p(y_i^{\text{pred}} = a | \mathbf{x}) &= p(y_i^{\text{pred}} = b | \mathbf{x}) \\ \Rightarrow \exp(w_a^T \mathbf{x}_i) &= \exp(w_b^T \mathbf{x}_i) \\ \Rightarrow (w_a^T - w_b^T) \mathbf{x}_i &= 0 \end{aligned}$$



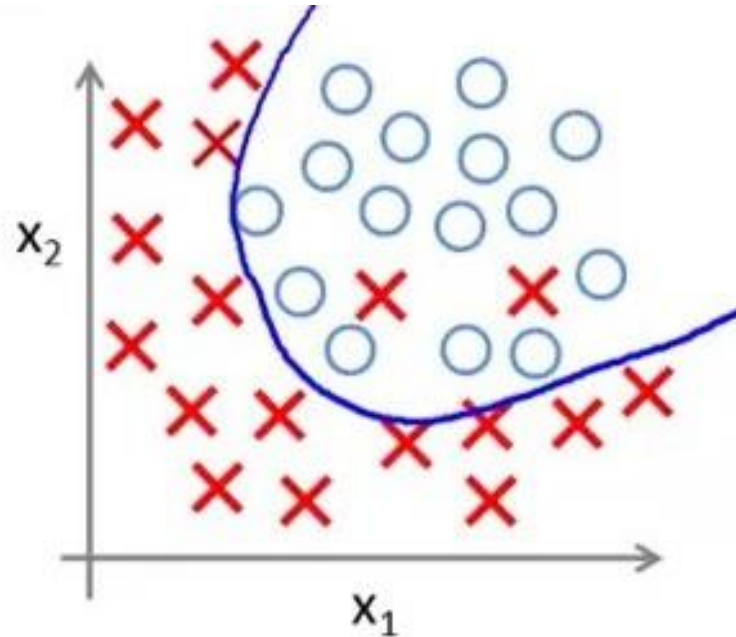
- Thus it is a linear boundary in features in $\mathbf{X}$.
- For 2-dim $\mathbf{x} = [x^1, x^2]$, it is the equation of a line: $x^1 = c x^2$ with $c = (w_a^T - w_b^T)$

Basis expansion changes the decision boundaries

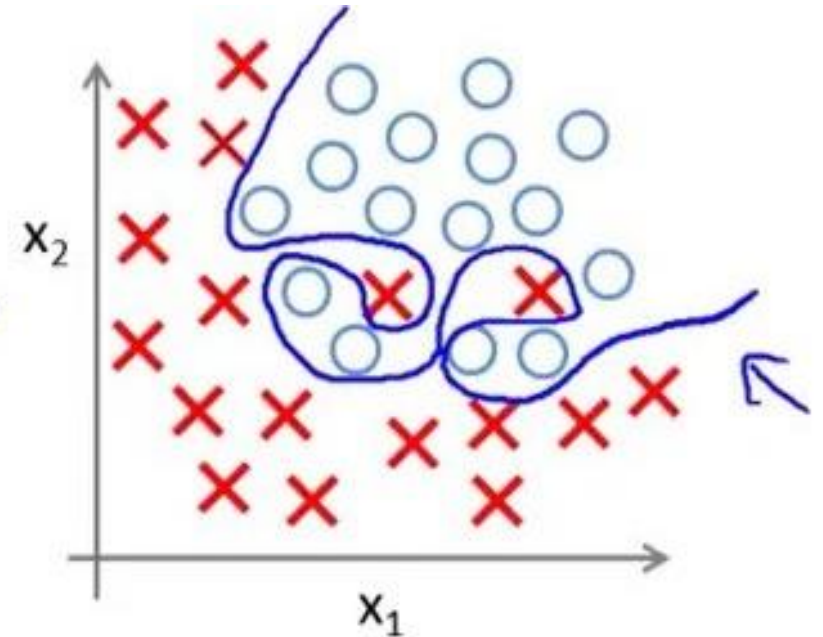


$\rightarrow h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$
(g = sigmoid function)

"Underfit"



$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2$
 $+ \theta_3 x_1^2 + \theta_4 x_2^2$
 $+ \theta_5 \underline{x_1 x_2})$



$g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2$
 $+ \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2$
 $+ \theta_5 \underline{x_1^2 x_2^3} + \theta_6 \underline{x_1^3 x_2} + \dots)$

"Overfit"

MLE for (Binary) Logistic Regression (WLOG, $y_i \in \{0,1\}$)

Likelihood is naturally written as a binomial with

$$p_1(x_i) = \textit{sigmoid}(w^T x_i), \text{ and } p_0(x_i) = 1 - p_1(x_i):$$

$$L(\{y_i\}|\{x_i\}, w^T) = p(\{y_i\}|w, \{x_i\}) = \prod_{i=1}^N p_1^{y_i}(x_i)(1 - p_1(x_i))^{1-y_i}$$

Thus the log likelihood is given by:

$$LL(\{y_i\}|\{x_i\}, w^T) = \sum_i^N y_i \ln(p_1(x_i)) + (1 - y_i) \ln(1 - p_1(x_i))$$

MLE for (Binary) Logistic Regression (WLOG, $y_i \in \{0,1\}$)

Take the derivative so that we can set it to zero:

$$LL(w) = \sum_{i=1}^n y_i \ln p_i + (1 - y_i) \ln(1 - p_i)$$

$$p_i = p(x_i) = \text{sigmoid}(w^T x_i) \\ = \sigma(w^T x_i)$$

$$\nabla_w LL(w) = \nabla_w \left(\sum_{i=1}^n y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right)$$

$$\frac{d}{da} \sigma(a) = \sigma(a)(1 - \sigma(a)) \\ = \sigma(a)\sigma(-a)$$

$$= \sum_{i=1}^n \frac{y_i}{p_i} \nabla_w p_i - \frac{1 - y_i}{1 - p_i} \nabla_w p_i$$

$$= \sum_{i=1}^n \left(\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i} \right) \nabla_w p_i = \sum_{i=1}^n \left(\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i} \right) p_i(1 - p_i) x_i$$

$$= \sum_{i=1}^n (y_i(1 - p_i) - (1 - y_i)(p_i)) x_i = \sum_{i=1}^n (y_i - p_i) x_i$$

In vector form: $\nabla_w LL(w) = -X^T (y - p)$ where $y, p \in \mathbb{R}^n$

MLE for (Binary) Logistic Regression

Is there a closed form solution for $\mathbf{w}$?

$$p_i = p(x_i) = \text{sigmoid}(\mathbf{w}^T \mathbf{x}_i) \\ = \sigma(\mathbf{w}^T \mathbf{x}_i)$$

$$\nabla_{\mathbf{w}} LL(\mathbf{w}) = -\mathbf{X}^T (\mathbf{y} - \mathbf{p}) \quad \text{where } \mathbf{y}, \mathbf{p} \in \mathbb{R}^n$$

No, so as with MoG (and soon neural networks), we need iterative optimization, such as gradient descent:

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \alpha_k \nabla_{\mathbf{w}} LL(\mathbf{w})$$

Likelihood for Softmax/Multinomial Regression

- We have a Multinomial likelihood, and so this approach is sometimes called “multinomial regression”:

$$L = p(\{y_i\}|\{w_k\}, \{x_i\}) = \prod_{i=1}^N \prod_{k=1}^K (p_i^k)^{[y_i=k]}$$

- where $p_i^k = \text{softmax}(k, \{w_k^T x_i\})$

- Log likelihood is:

$$LL = \sum_i^N \sum_k^K [y_i=k] \log(p_i^k)$$

- (rest is analogous to binary case—won't do here)

CS 189/289

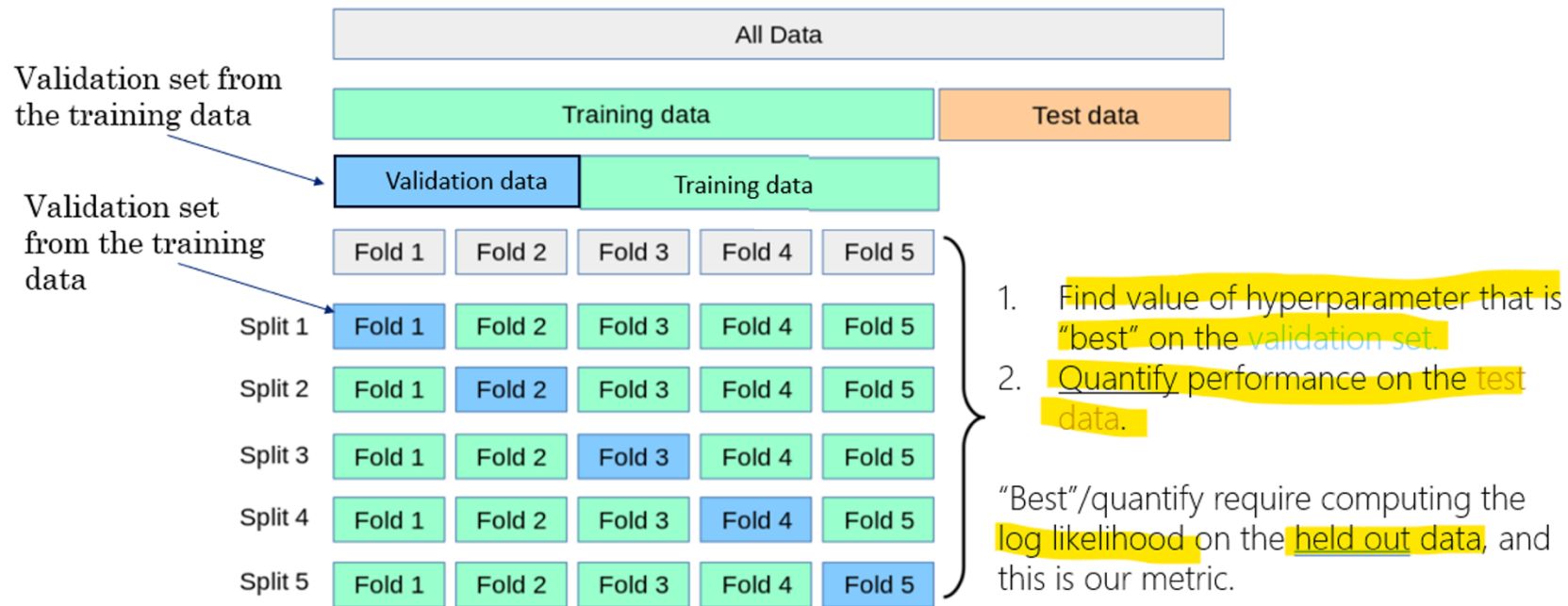
Today's lecture outline:

1. Logistic Regression
2. Evaluating Classifiers

Metrics for Classification

$$p(Y = k|X)$$

K-fold cross-validation



What evaluation metric might we compute for each test fold?

How to choose between two classifiers?

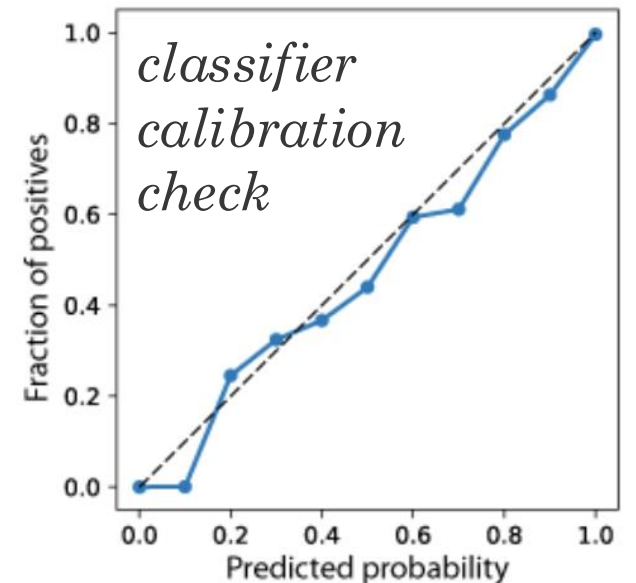
TRUTH	Logistic regression	Neural network
1	0.7198	0.9038
0	0.2460	0.8455
0	0.1219	0.4655
0	0.1560	0.3204
0	0.7527	0.2491
1	0.3064	0.7129
0	0.7194	0.4983
0	0.5531	0.6513
1	0.2173	0.3806
0	0.0839	0.1619
1	0.8429	0.7028

What evaluation metric/quantity applied to these data could help decide which model to use?

How to pick between these two classifiers?

- Could use likelihood.
- Could use threshold of $p=0.5$ and count the # of misclassifications ("accuracy").
- Useful if model is *calibrated* (probabilities mean what they say).

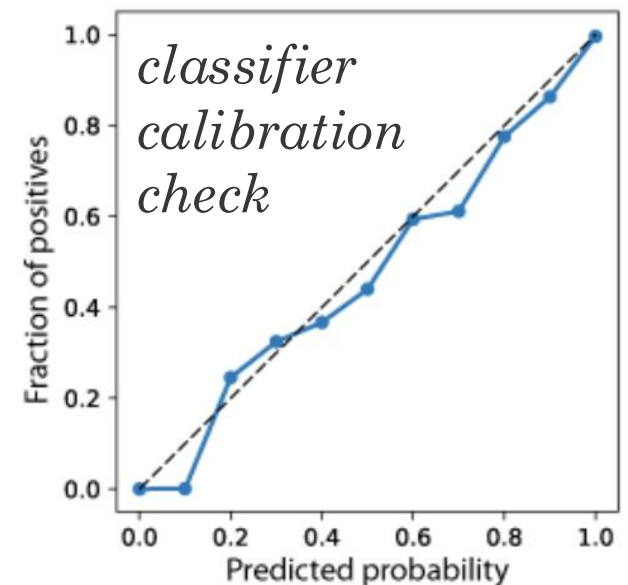
TRUTH	Logistic regression	Neural network
1	0.7198	0.9038
0	0.2460	0.8455
0	0.1219	0.4655
0	0.1560	0.3204
0	0.7527	0.2491
1	0.3064	0.7129
0	0.7194	0.4983
0	0.5531	0.6513
1	0.2173	0.3806
0	0.0839	0.1619
1	0.8429	0.7028



How to pick between these two classifiers?

- Could use likelihood.
- Could use threshold of $p=0.5$ and count the # of misclassifications ("accuracy").
- Useful if model is *calibrated* (probabilities mean what they say).
- "ROC curves" are going to help us deal with these issues (also general "scoring", non-probabilistic classifiers).

TRUTH	Logistic regression	Neural network
1	0.7198	0.9038
0	0.2460	0.8455
0	0.1219	0.4655
0	0.1560	0.3204
0	0.7527	0.2491
1	0.3064	0.7129
0	0.7194	0.4983
0	0.5531	0.6513
1	0.2173	0.3806
0	0.0839	0.1619
1	0.8429	0.7028



Case study: a miscalibrated but useful classifier

TRUTH	Logistic	Neural network
1	0.88	0.9038
0	0.77	0.8455
0	0.59	0.4655
0	0.81	0.3204
0	0.7527	0.2491
1	0.93	0.63
0	0.7194	0.4983
0	0.5531	0.6513
1	0.98	0.3806
0	0.0839	0.1619
1	0.8429	0.7028

- If we pick the “optimal” decision of $p=0.5$ decision boundary, we will choose the model “containing less information” (NN)!

Case study: a miscalibrated but useful classifier

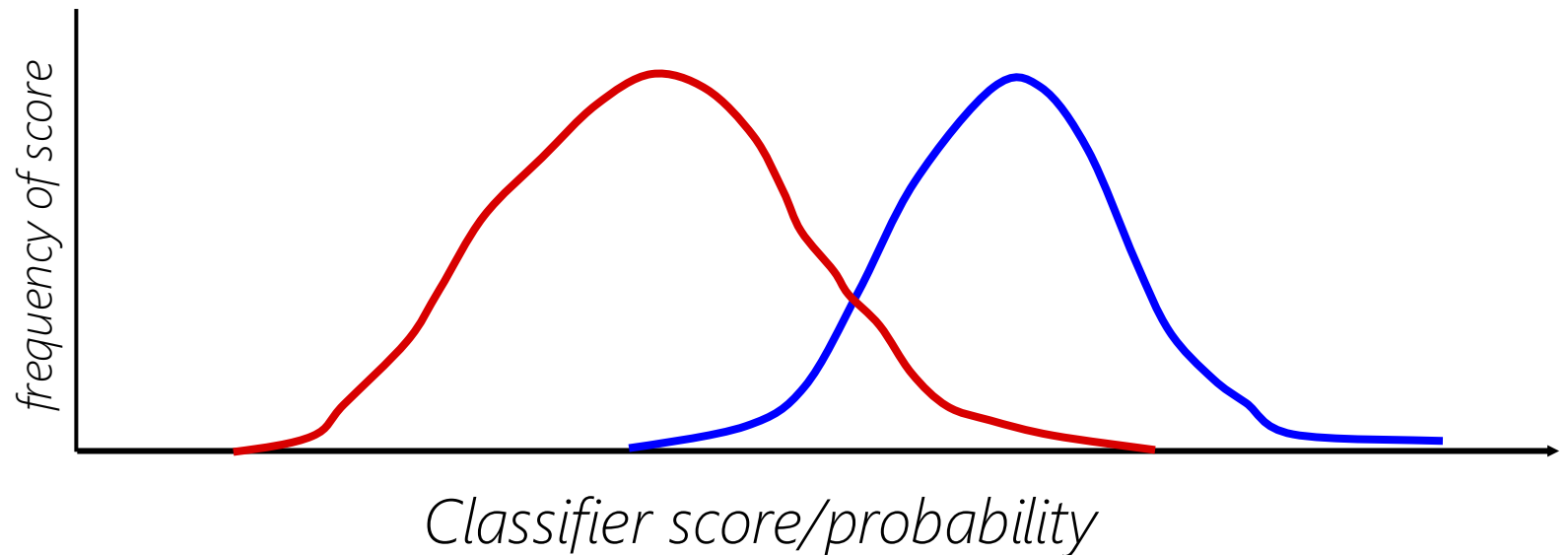
TRUTH	Logistic	Neural network
1	0.88	0.9038
0	0.77	0.8455
0	0.59	0.4655
0	0.81	0.3204
0	0.7527	0.2491
1	0.93	0.63
0	0.7194	0.4983
0	0.5531	0.6513
1	0.98	0.3806
0	0.0839	0.1619
1	0.8429	0.7028

- If we pick the “optimal” decision of $p=0.5$ decision boundary, we will choose the model “containing less information” (NN)!
- Whereas best LR threshold of 0.8 gives 0 errors, while best NN threshold of 0.5 gives 3 errors.
- Model knows to assign the true “1”s with a higher “score” than the true “0”s.

Defining false positives, false negatives, etc.

[We will consider only binary classifiers in today's lecture]

*Distribution of classifier "scores" of
unhealthy and healthy individuals
in a test set*

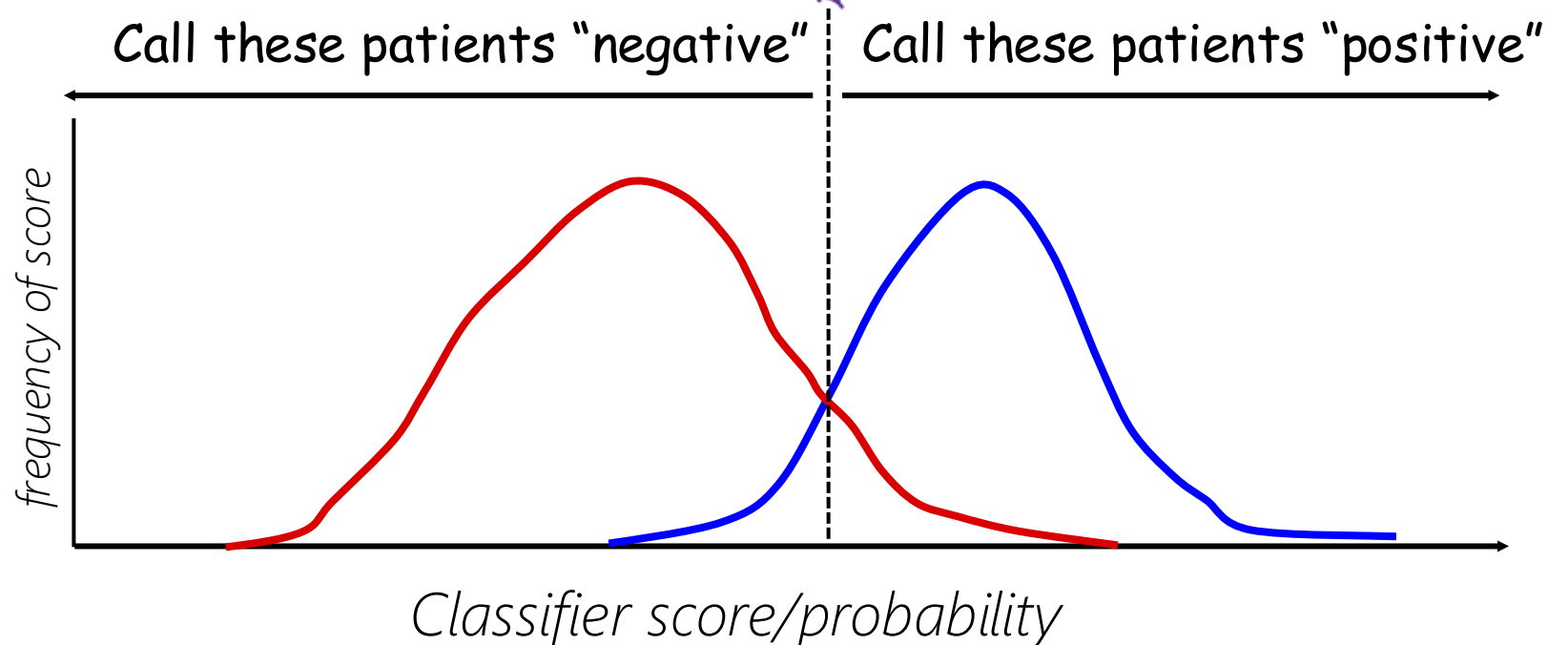


Defining false positives, false negatives, etc.

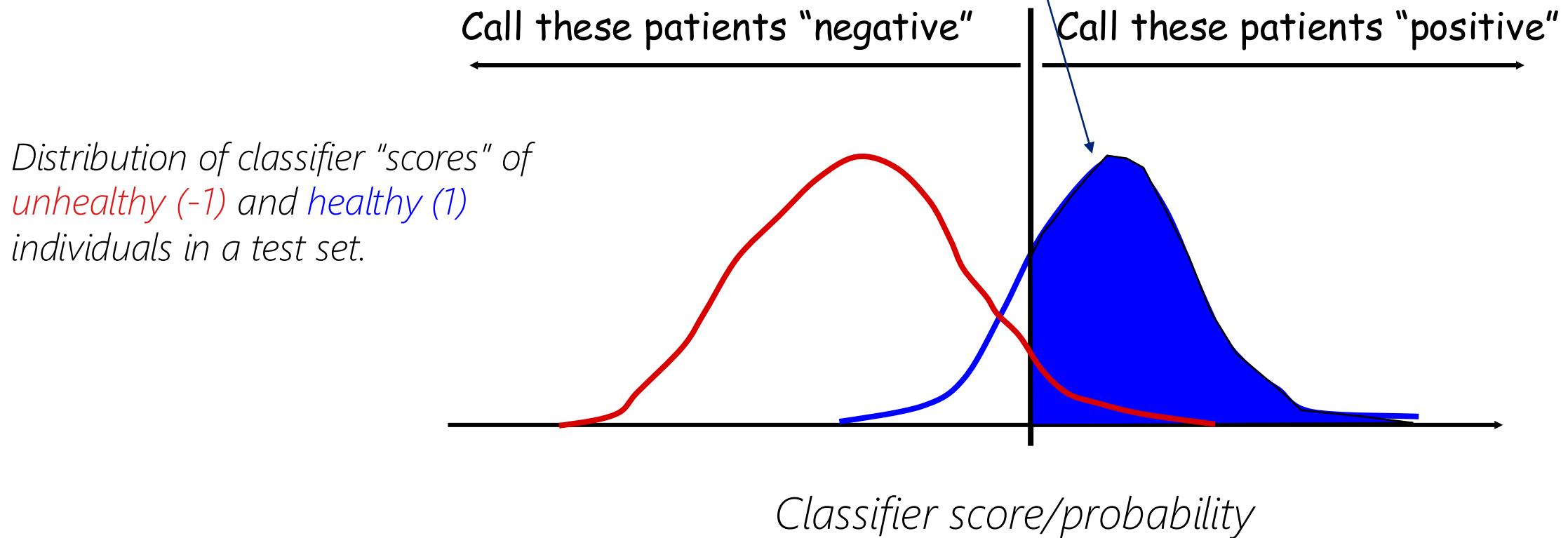
[We will consider only binary classifiers in today's lecture]

Choose a threshold on the score/probabilistic output, and call/predict all samples above it a "1" (e.g. "healthy") and all those below it a "-1" (e.g. "unhealthy").

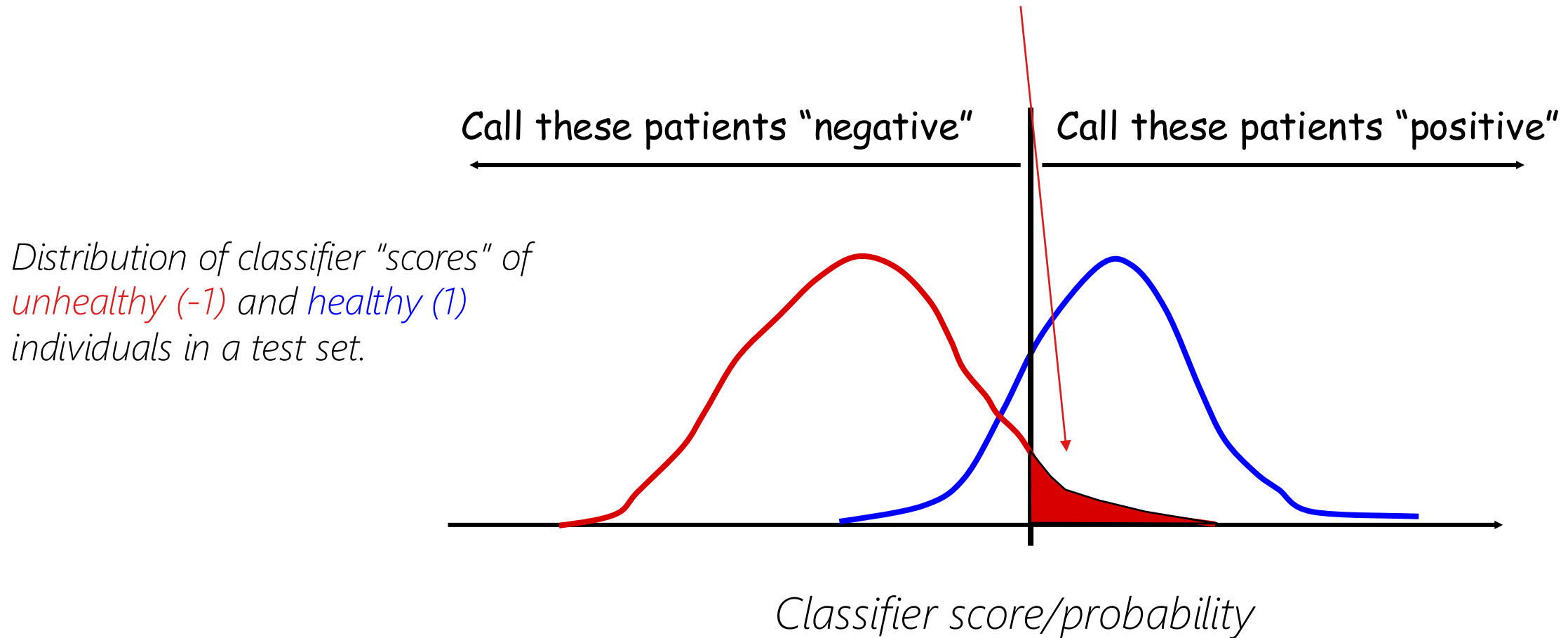
*Distribution of classifier "scores" of
unhealthy and healthy individuals
in a test set*



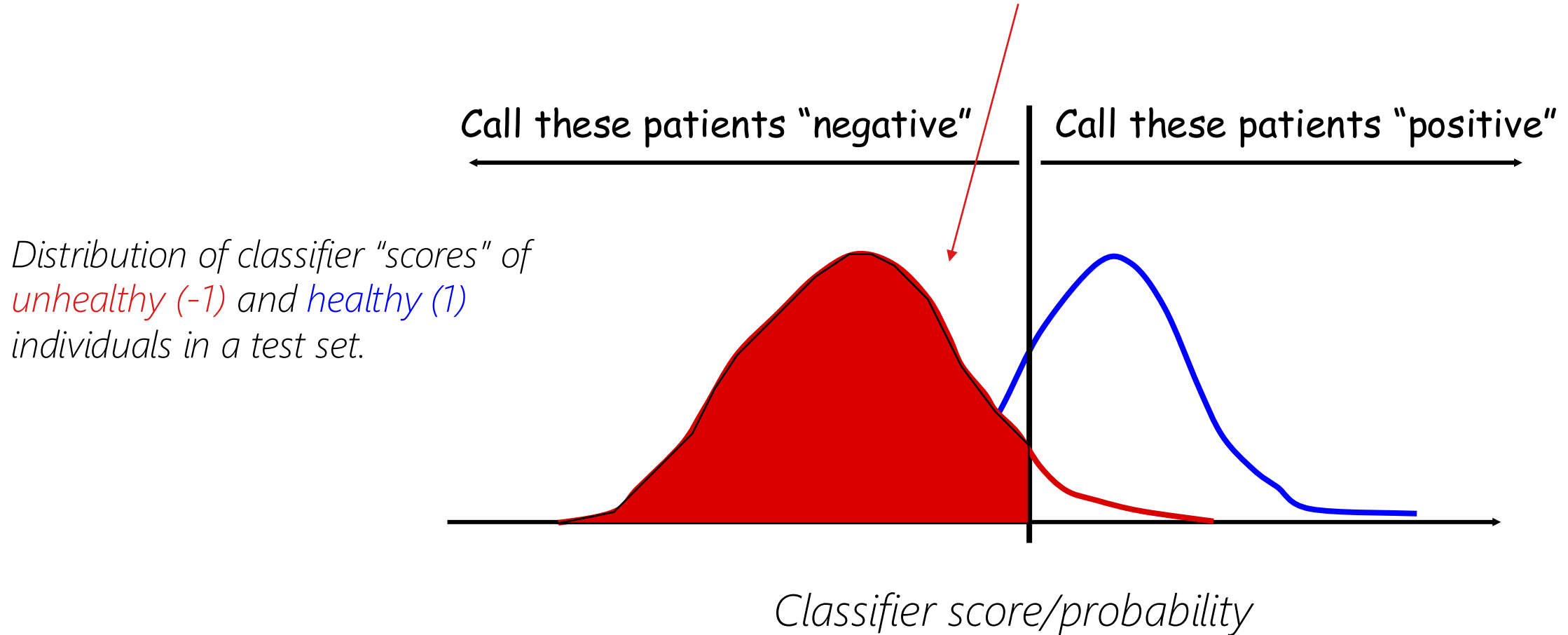
Definitions: True Positives (TP)



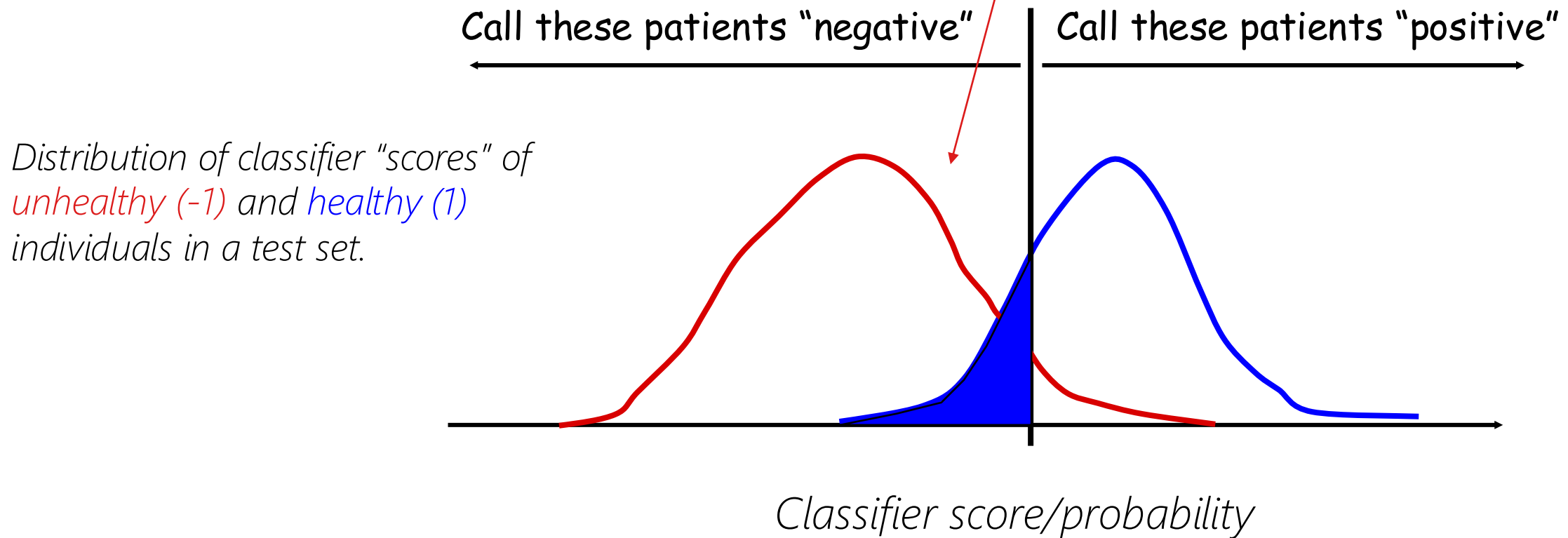
Definitions: False Positives (FP)



Definitions: True Negatives (TN)



Definitions: False Negatives (FN)

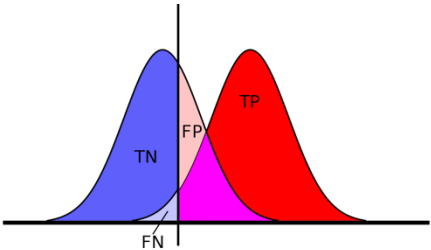


Summary of classifier decision outcomes

$$accuracy(threshold) = \frac{TP + TN}{TP + TN + FP + FN} = \frac{TP + TN}{N}$$

Confusion matrix:

		MODEL PREDICTIONS	
		Negative	Positive
GROUND TRUTH	Negative	TN	FP
	Positive	FN	TP



TP	FP
FN	TN

#actual negatives = TN + FP

#actual positives = FN + TP

Pitfalls of Accuracy: A Case Study

$$\begin{aligned} & \text{accuracy}(\text{threshold}) \\ & \quad \frac{TP + TN}{TP + TN + FP + FN} \end{aligned}$$

Suppose we're trying to build a classifier to filter spam emails.

Each email is **spam** (1) or **ham** (0).

We have 100 emails: 5 of them are **spam**, and the remaining 95 are real (i.e., **ham**).

Classifier 1:

Classify every email as **ham** (0).

$$\text{accuracy}_1 = \frac{95}{100} = 0.95$$

High accuracy...

...but we detected **none** ⚠ of the spam!!!

Classifier 2:

Classify every email as **spam** (1).

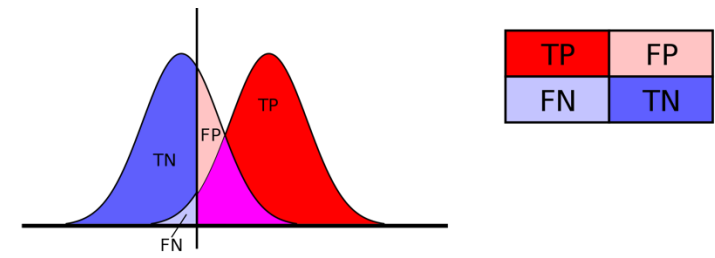
$$\text{accuracy}_2 = \frac{5}{100} = 0.05$$

Low ⚠ accuracy...

...but we detected **all** of the spam!!!

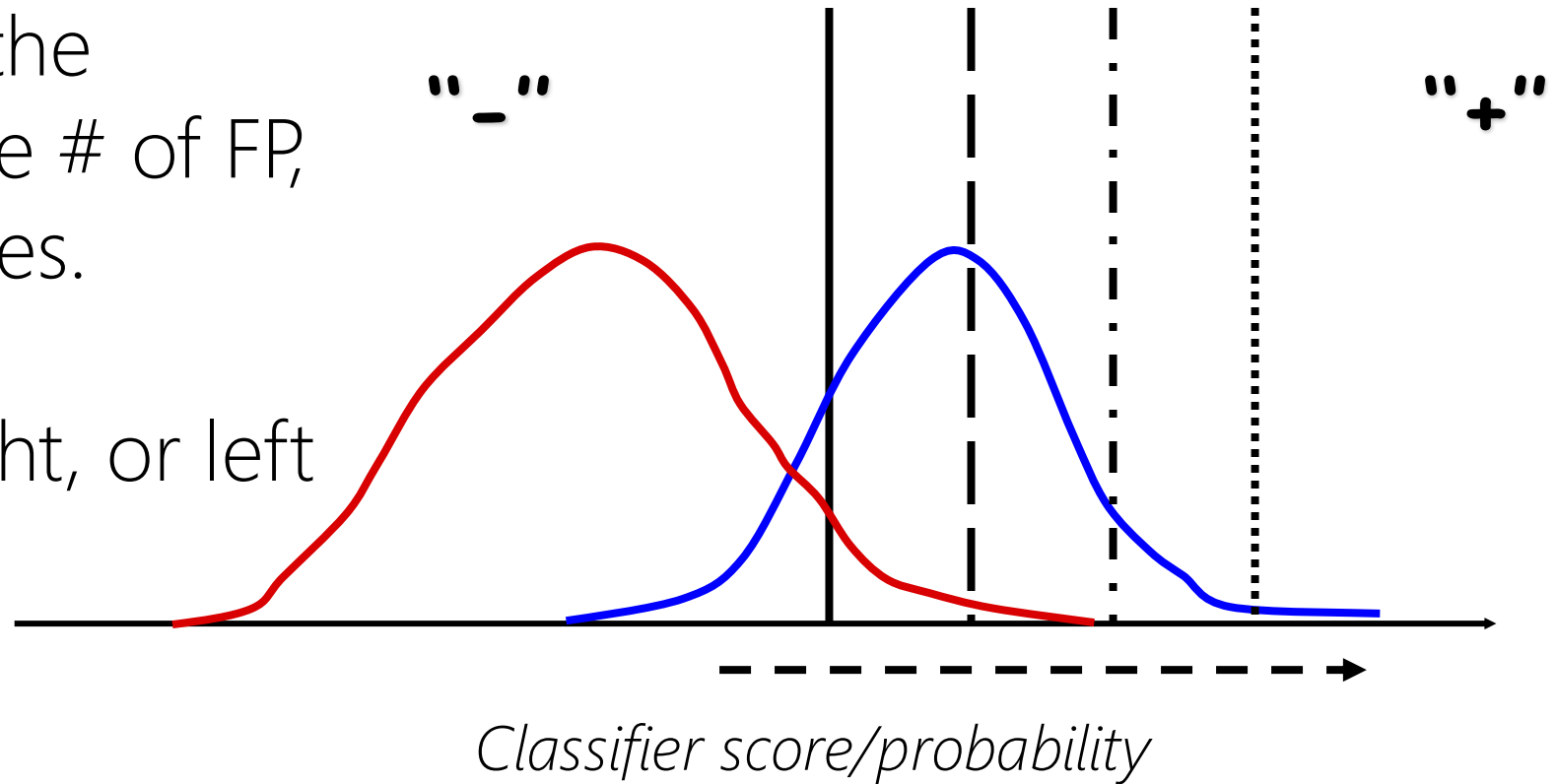
- While widely used, the accuracy metric is **is problematic** if there is **class imbalance**.
- Also, what threshold do we use?

Back to our decision threshold

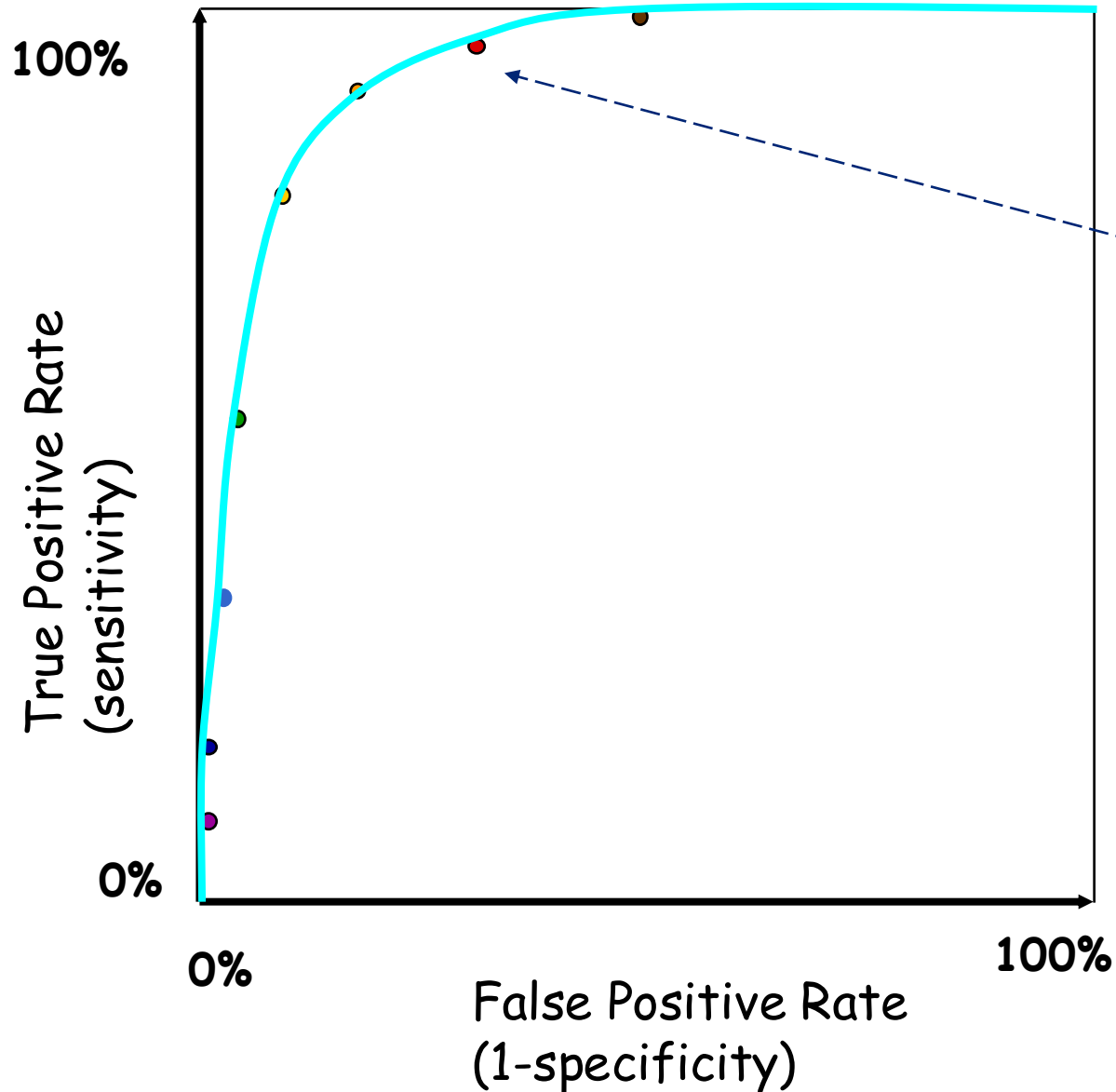


Every time we move the decision threshold, the # of FP, FN, TP and TN changes.

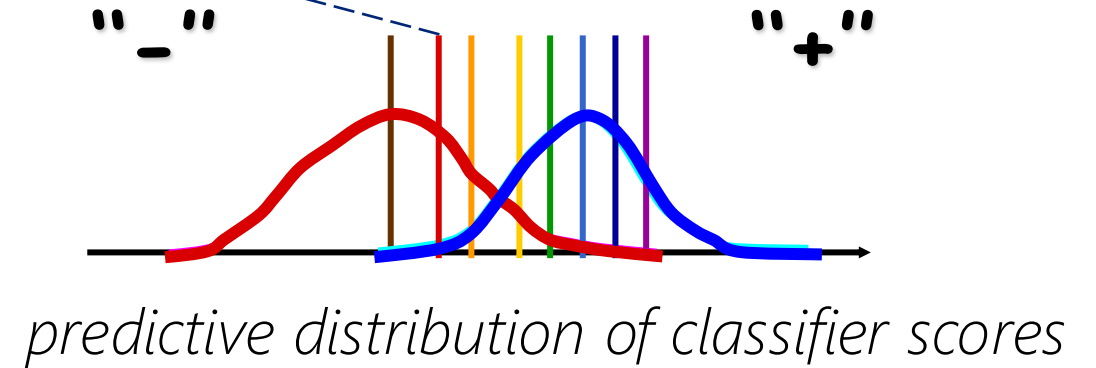
e.g. shifting to the right, or left



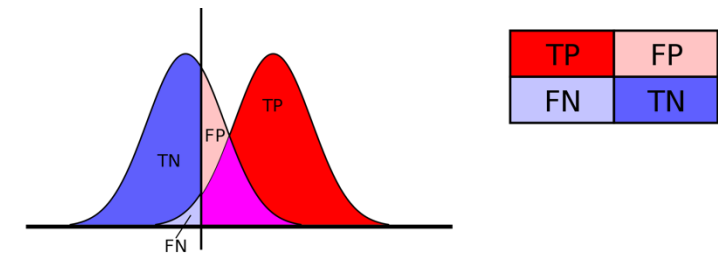
As we shift it, we can draw out an *ROC curve*



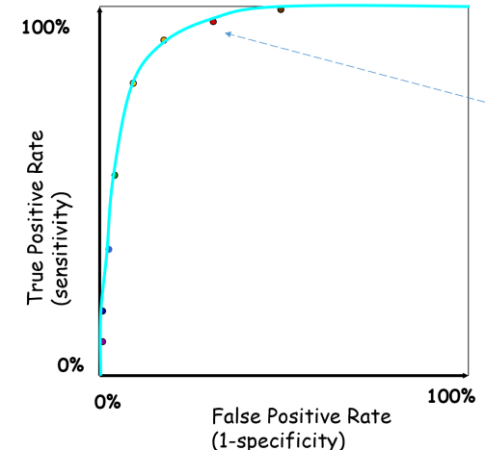
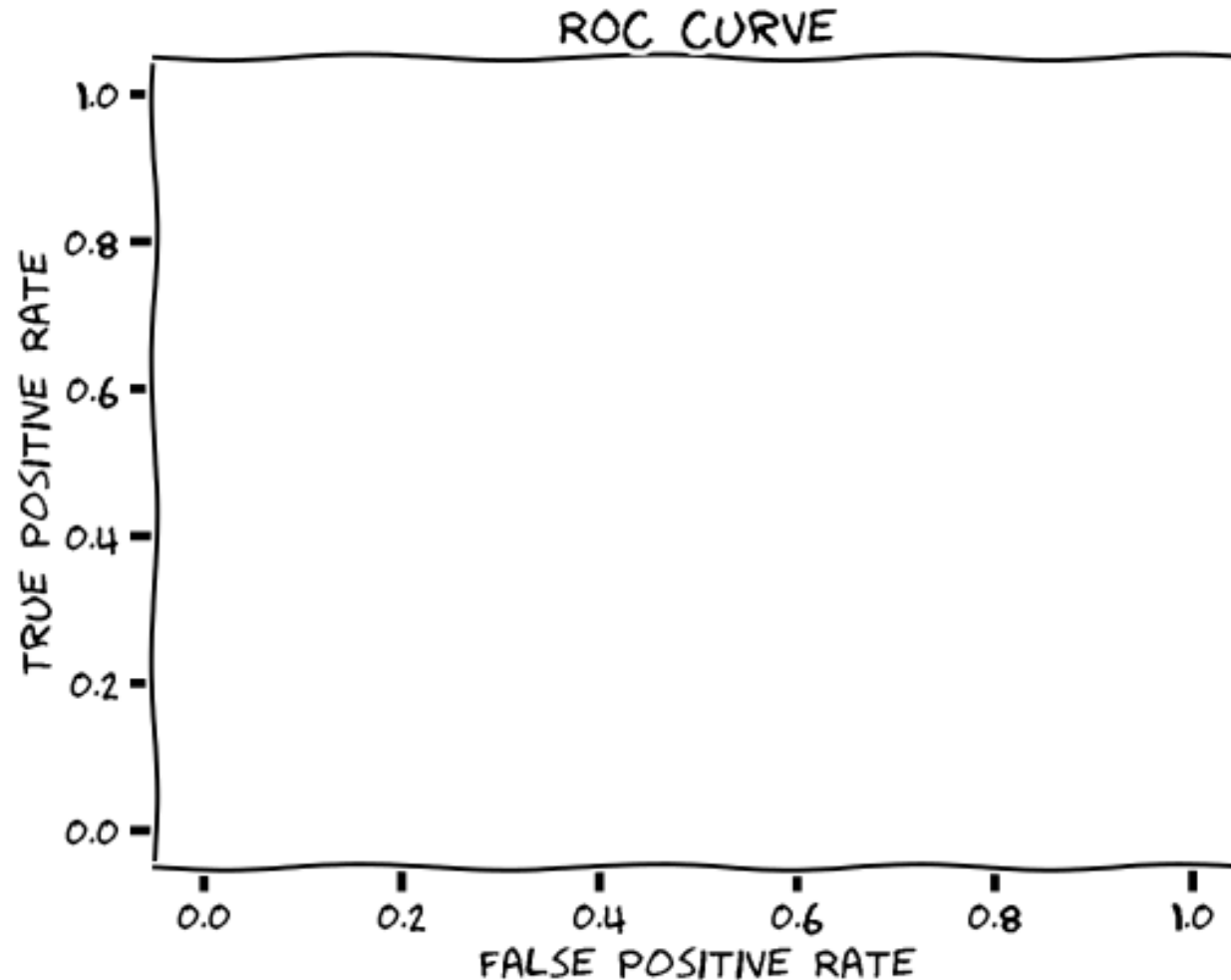
Receiver Operating Characteristic (ROC) curve



predictive distribution of classifier scores

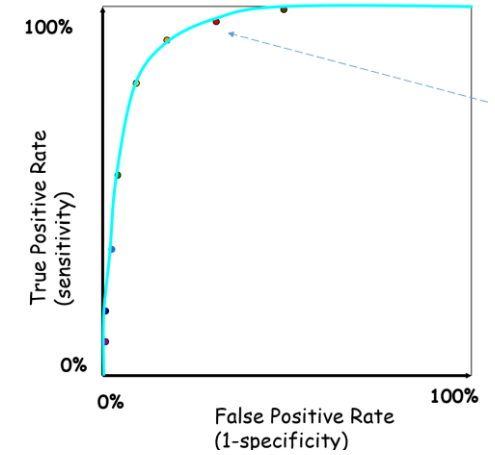
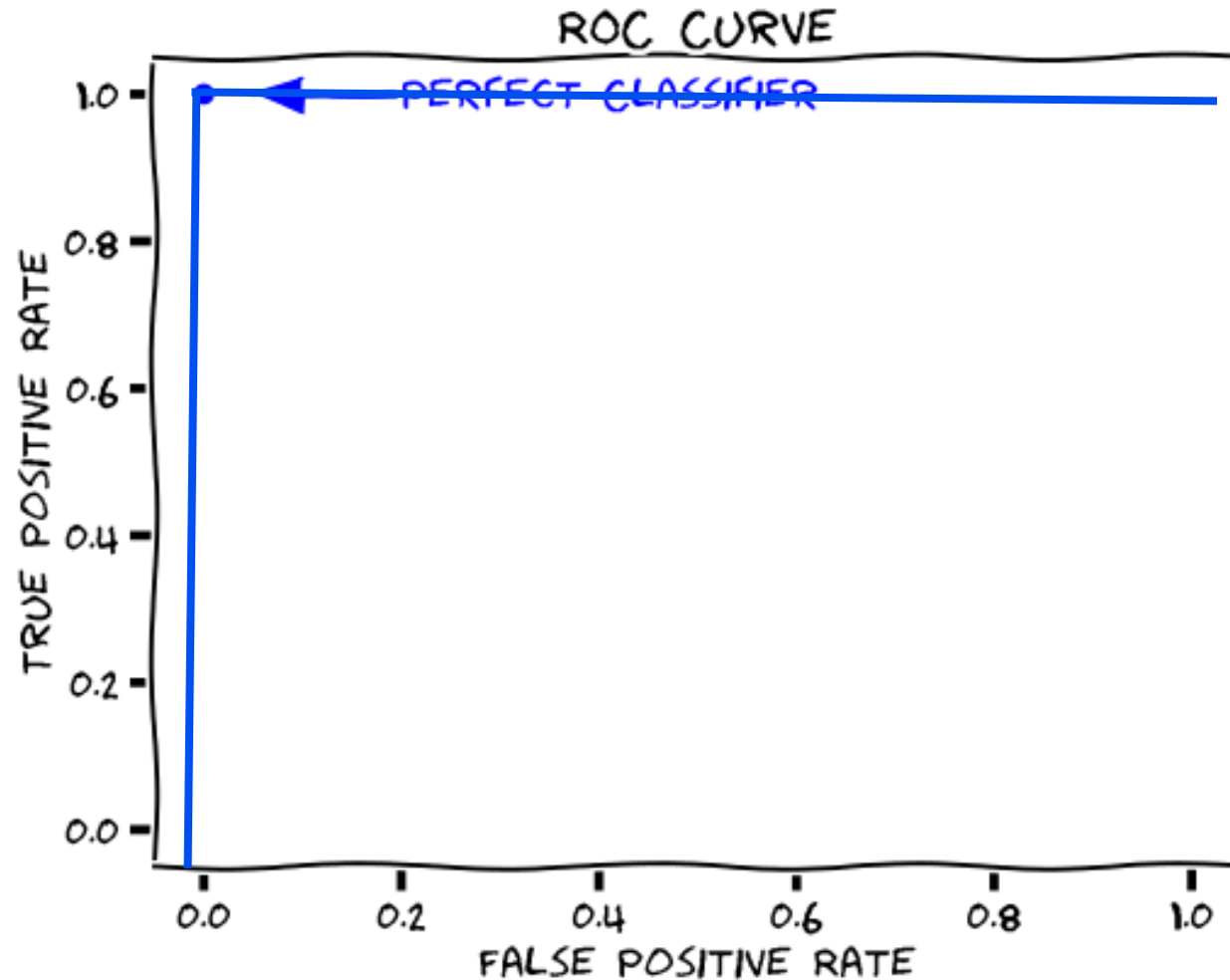


Comparing ROC curves across classifiers

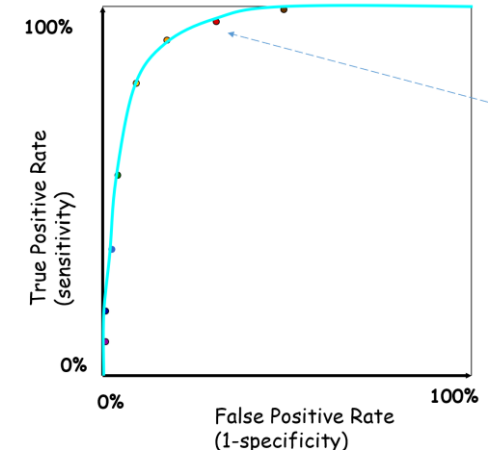
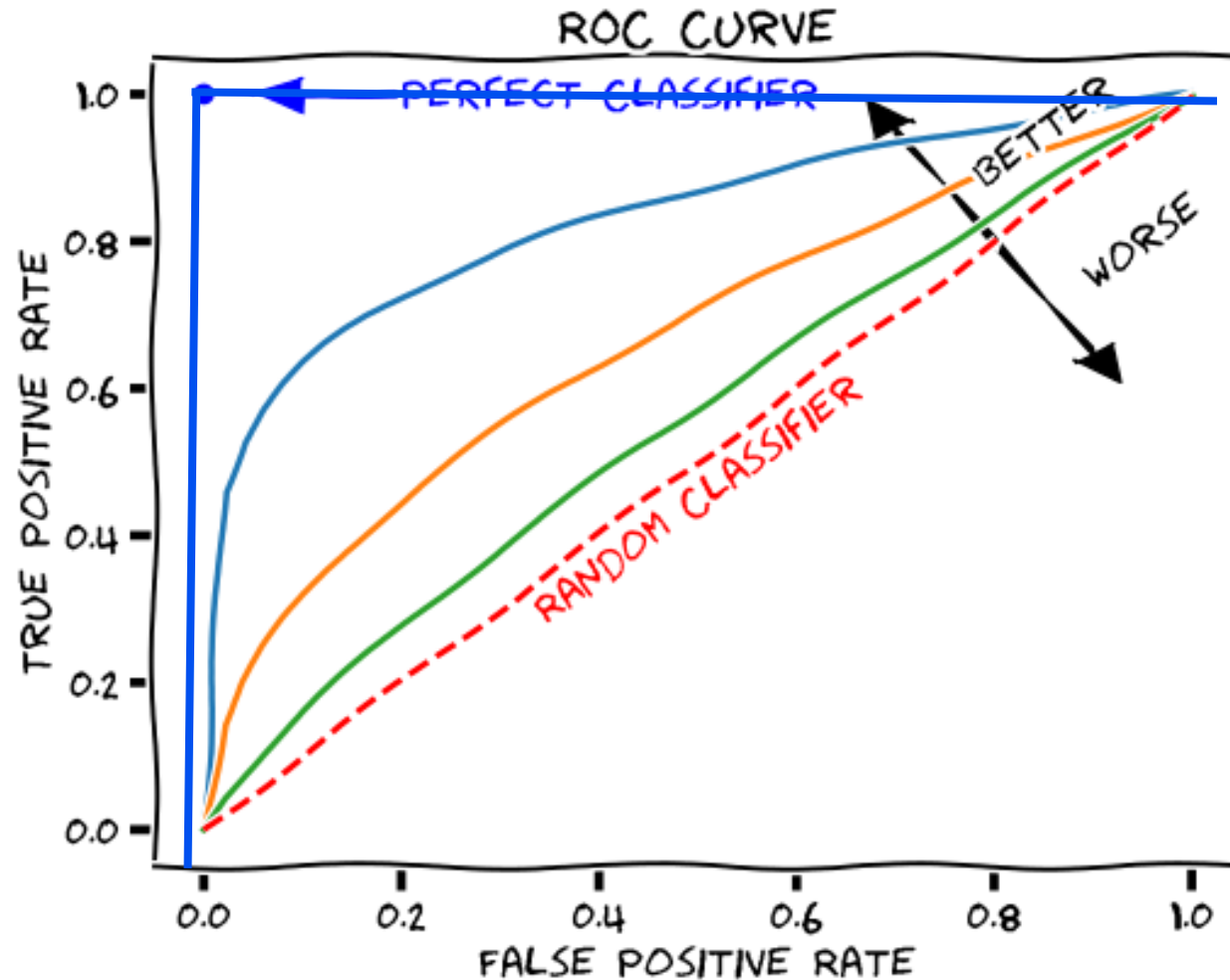


How would a perfect classifier appear on this plot?

Comparing ROC curves across classifiers

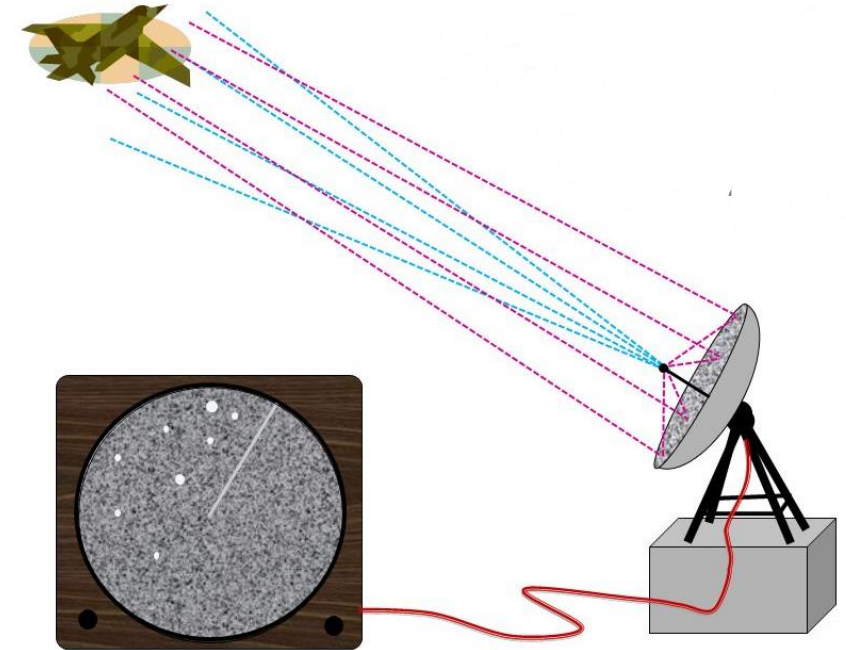


Comparing ROC curves across classifiers

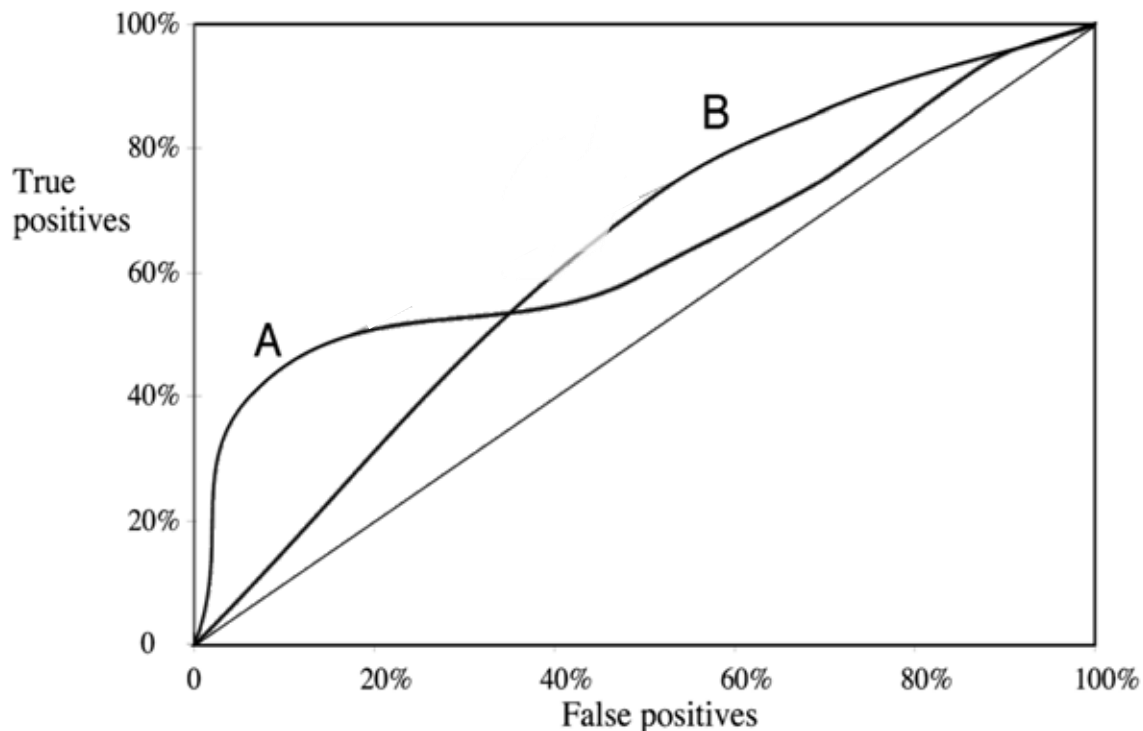


History of ROC curves

- "The ROC curve was first developed by electrical engineers and radar engineers during World War II (1939-45) for detecting enemy objects in battle fields and was soon introduced to psychology to account for perceptual detection of stimuli."
- "ROC analysis since then has been used in medicine, radiology, biometrics, and other areas for many decades and often used in machine learning and data mining research and applications."



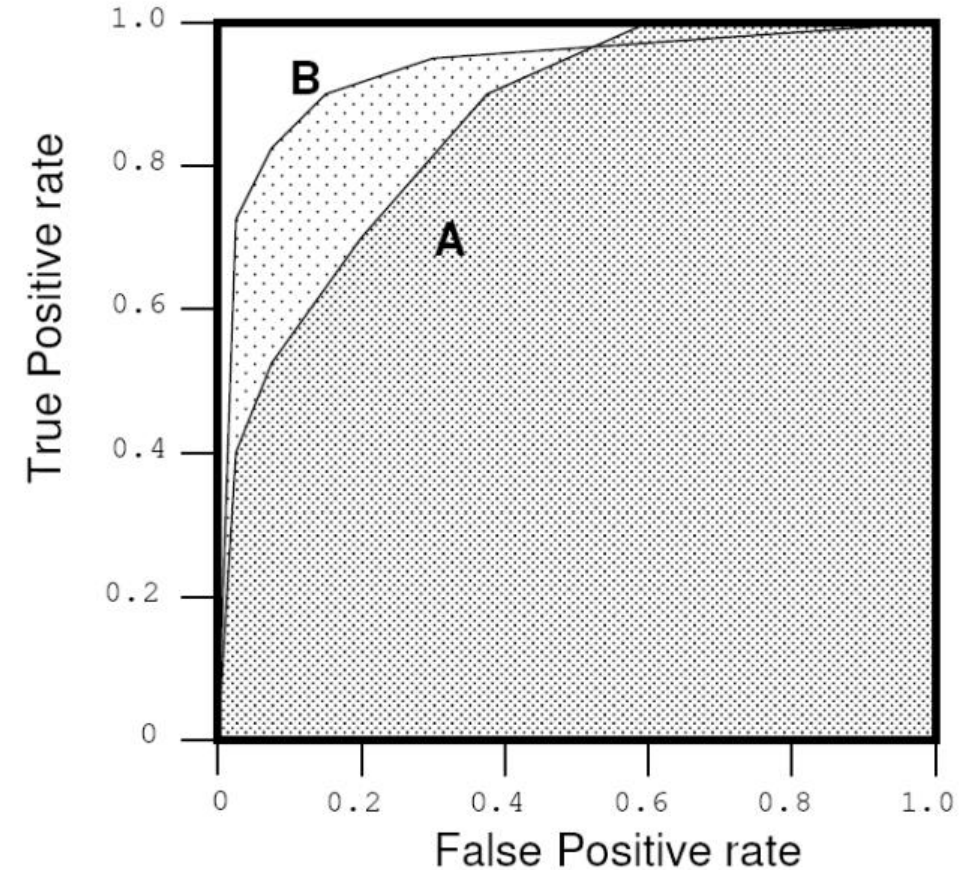
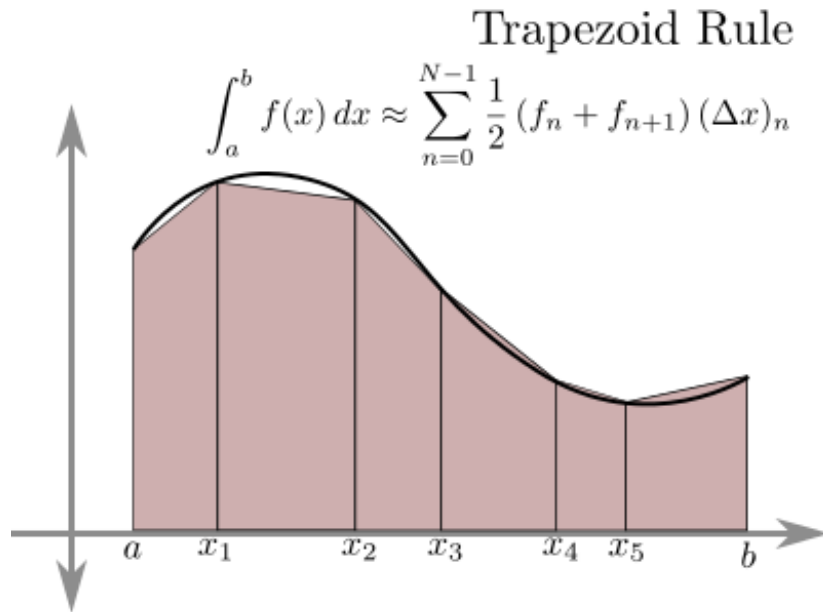
Which model would you choose?



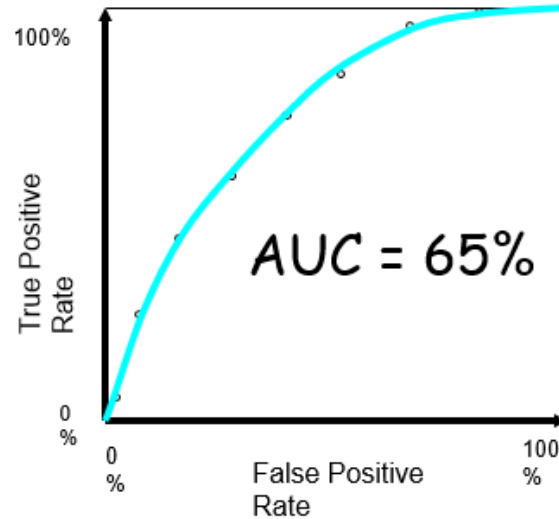
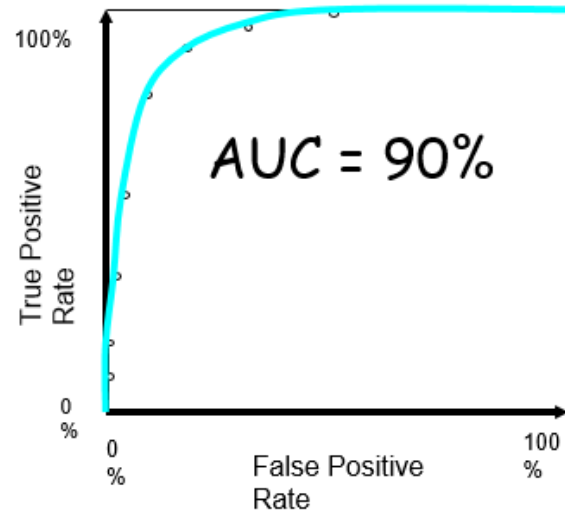
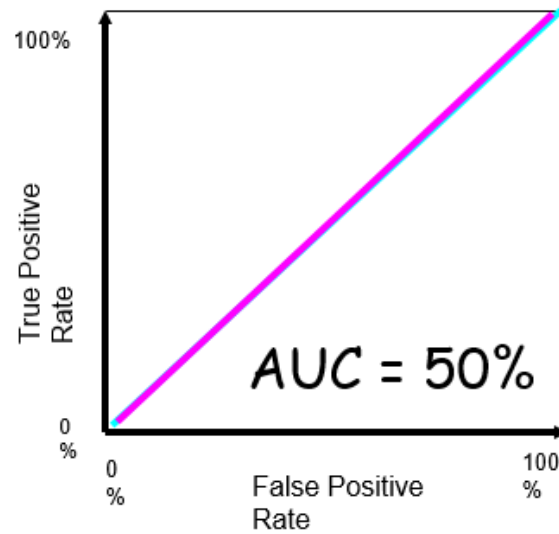
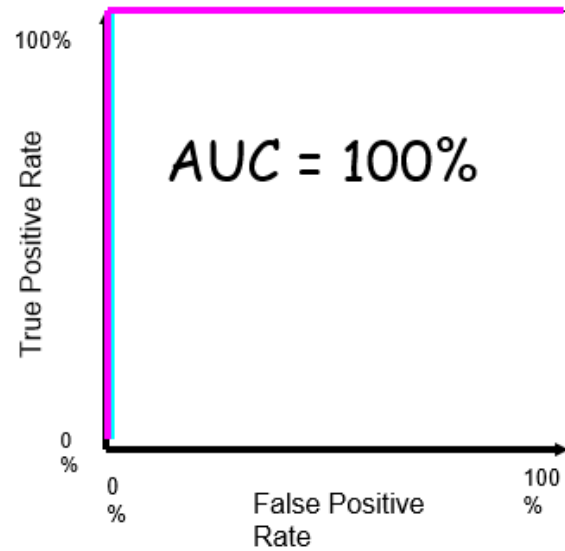
- In this example, no one classifier (A or B) *dominates* in ROC space.
- Thus we might ask what regime is most relevant to our problem.
- *e.g.* suppose you know you need few false positives, then you should pick method A, which dominates **up until FPR 40%**.

Summarizing ROCs with the Area Under the Curve (AUC)

- AUC: often used to compare classifiers.
- The bigger the AUC the better.



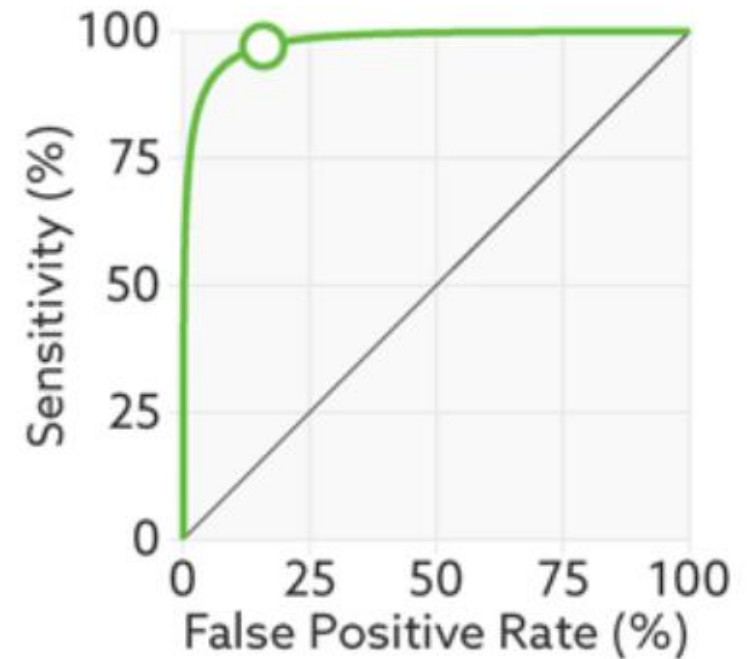
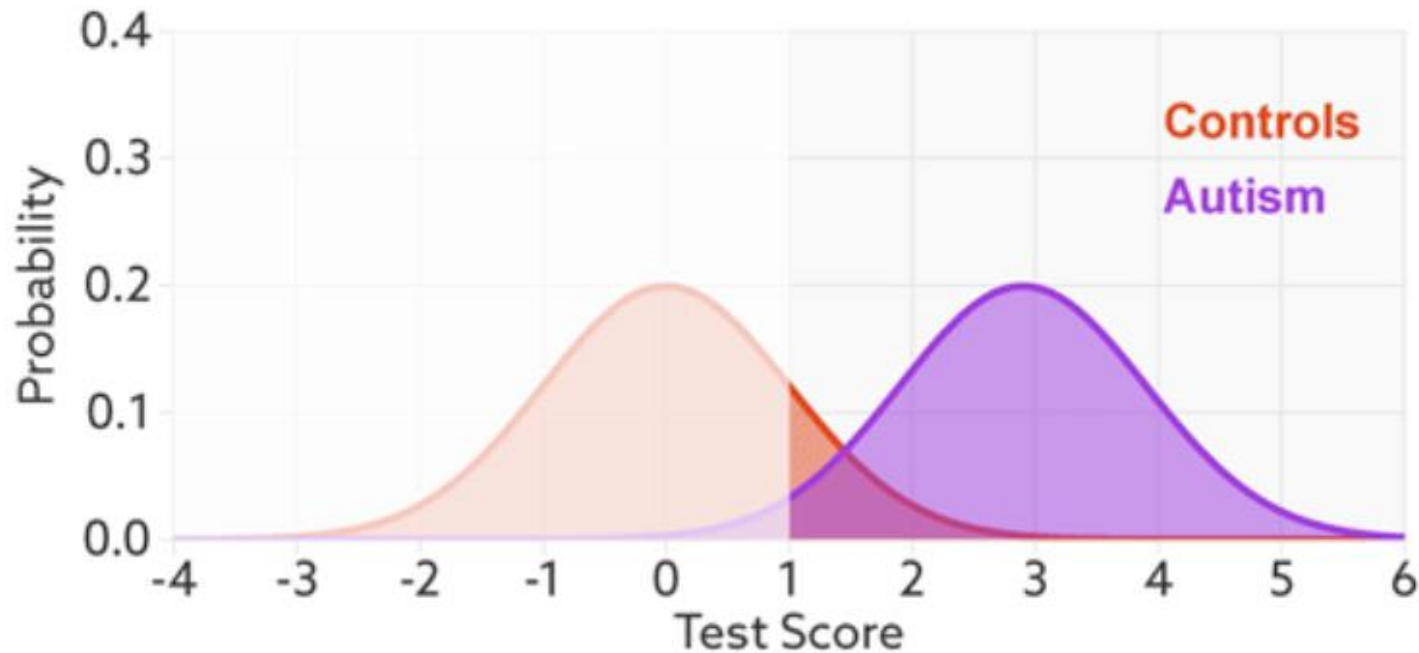
Summarizing ROCs with the Area Under the Curve (AUC)



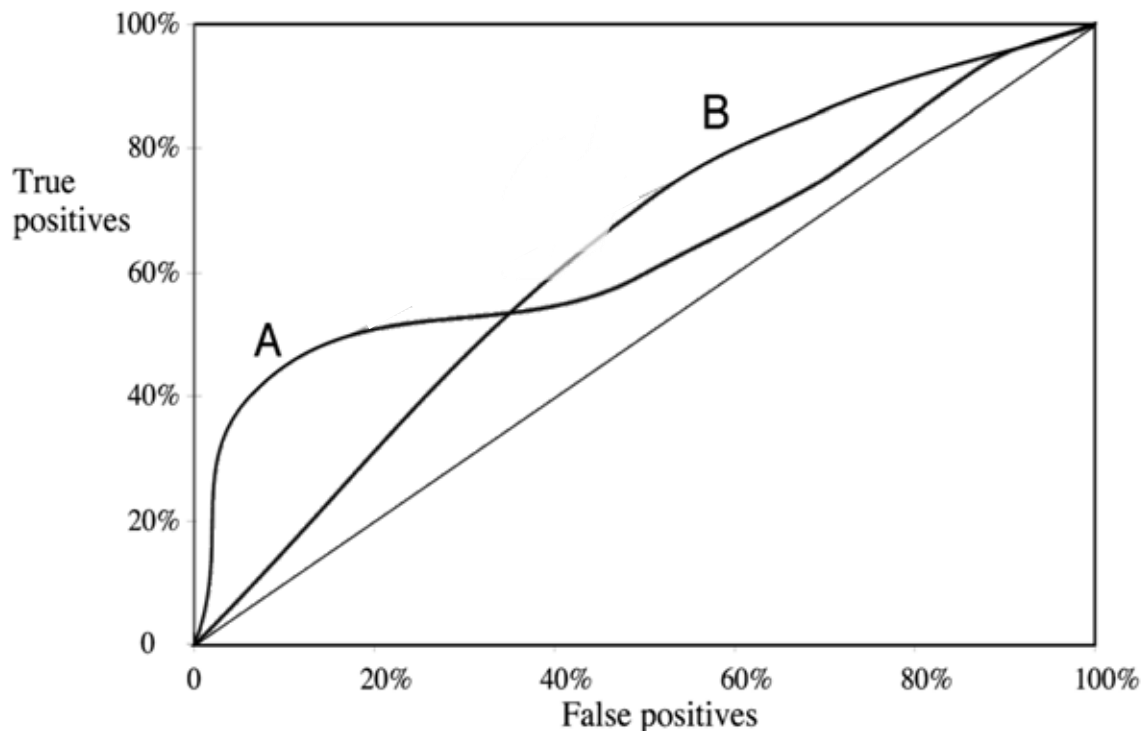
The AUC of a classifier is equivalent to the probability that the classifier will rank a randomly chosen positive sample higher than a randomly chosen negative sample.

Visualization of score distributions wrt ROCs & AUC

The more separated the data are by the classifier scores, the larger the AUC.



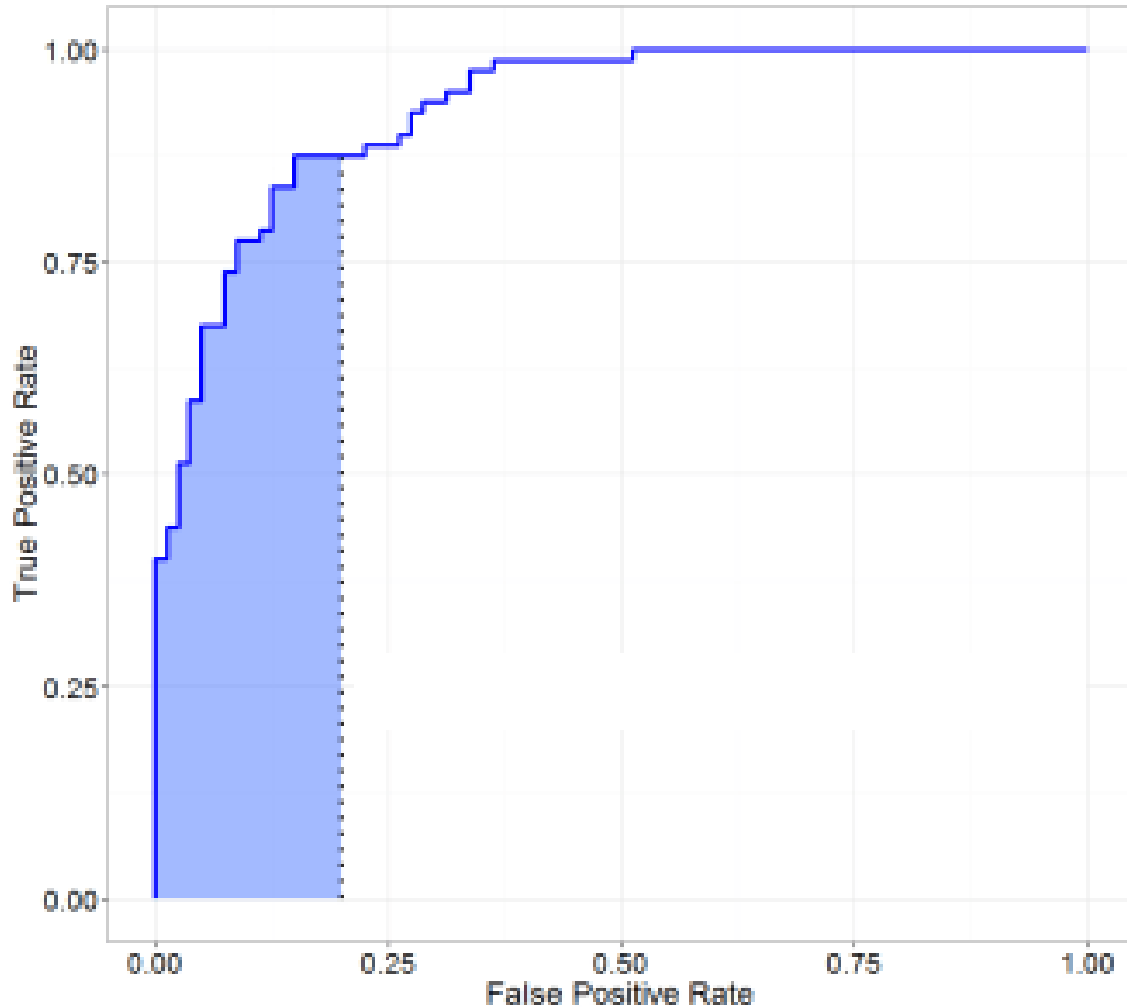
Recall: Which model would you choose?



- In this example, no one classifier (A or B) *dominates* in ROC space.
- Thus we might ask what regime is most relevant to our problem.
- *e.g.* suppose you know you need few false positives, then you should pick method A, which dominates **up until FPR 40%**.

partial Area Under the Curve

Partial Area Under the Curve (AUC)



Sometimes we know we would never operate with a FPR above some amount, and so we compute a *partial* AUC, here $AUC(0.2)$.

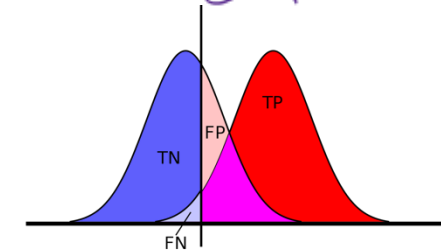
- e.g. deciding on medical intervention like chemotherapy
- e.g. deciding to spend lots of \$\$\$ in follow up biology experiments.

Summary of classifier decision outcomes (τ)

Rates: "normalize" by #samples who could have had that call:

- TP rate, $\text{TPR} = \text{TP} / \# \text{actual positives} = \text{TP} / (\text{FN} + \text{TP})$, aka *Sensitivity*
- TN rate, $\text{TNR} = \text{TN} / \# \text{actual negatives} = \text{TN} / (\text{TN} + \text{FP})$, aka *Specificity*
- *"Precision"* = $\text{TP} / (\# \text{predicted positive}) = \text{TP} / (\text{TP} + \text{FP})$ —this now depends on class distribution in set of points being evaluated.
- *"Recall"* = $\text{TPR} = \text{Sensitivity}$.

		MODEL PREDICTIONS	
		Negative	Positive
GROUND TRUTH	Negative	TN	FP
	Positive	FN	TP



TP	FP
FN	TN

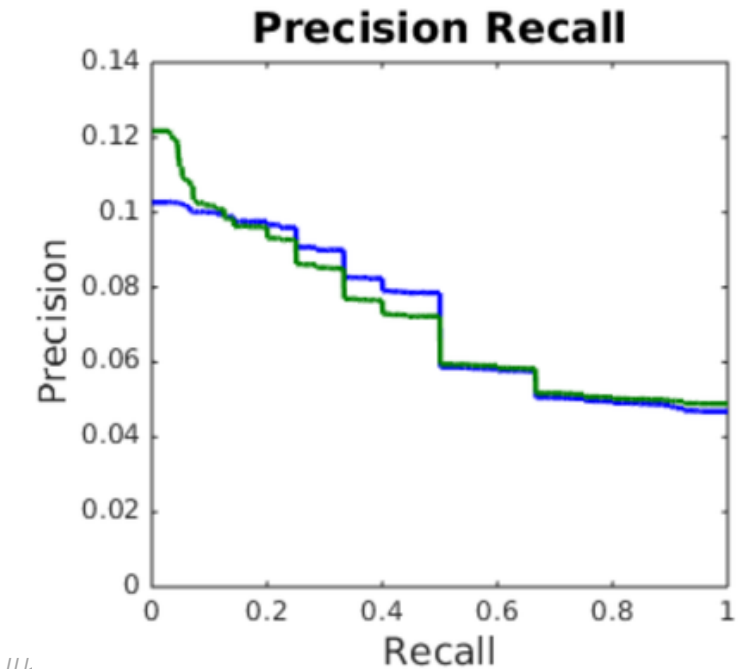
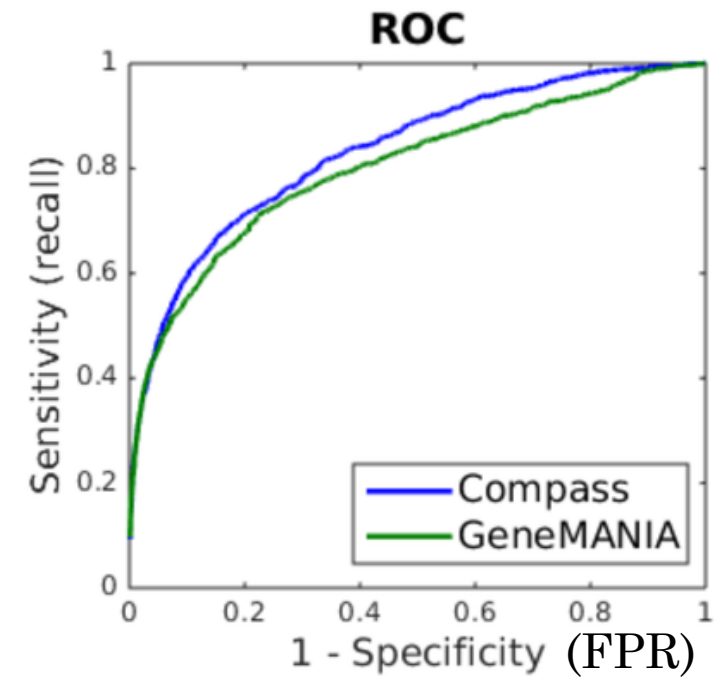
#actual negatives = $\text{TN} + \text{FP}$

#actual positives = $\text{FN} + \text{TP}$

Precision Recall Curves

Wrt ROC: replace FPR with Precision and flip the axes.

- ROC curves useful when we want invariance to class distribution.
- Analog to ROC-AUC is the AUPR.
- Precision-recall curves useful when care about (and know) the balance of the classes at test time.



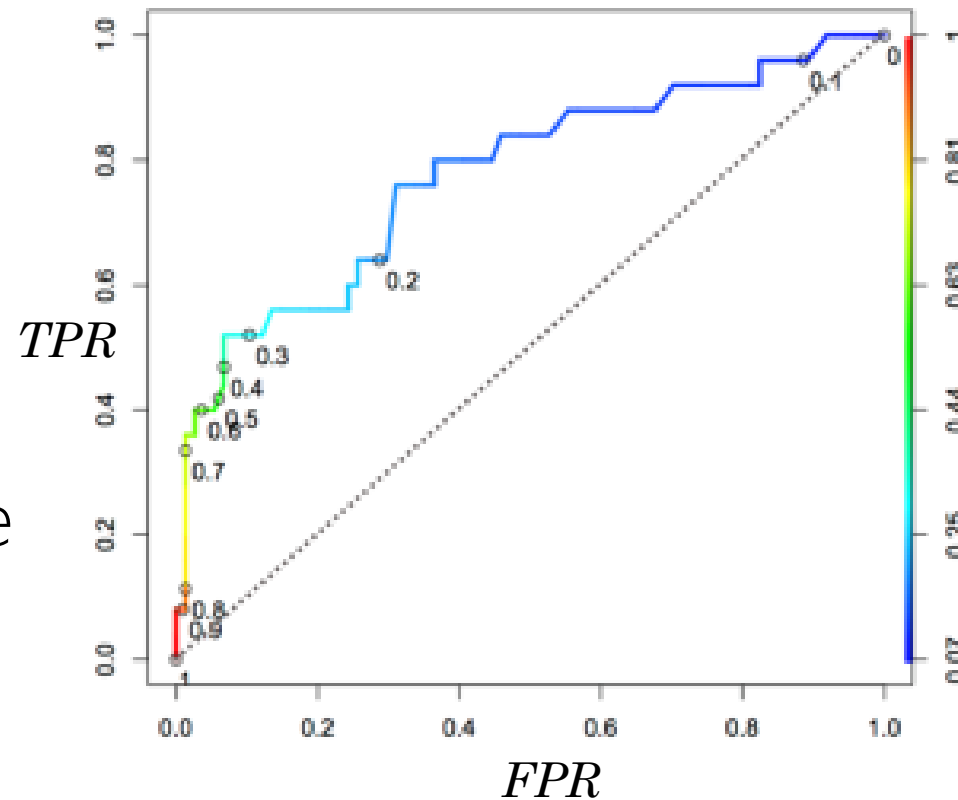
More slides, not responsible for this material unless covered elsewhere

An algorithm for making an ROC curve

Exploit the property that any instance that is classified as + at a given threshold, will be classified as + for all lower thresholds as well:

1. Sort the test instances by decreasing score.
2. Move down the list (lowering the threshold), processing one instance at a time. For each instance,
3. Computer the TPR and FPR and add one point to the plot.

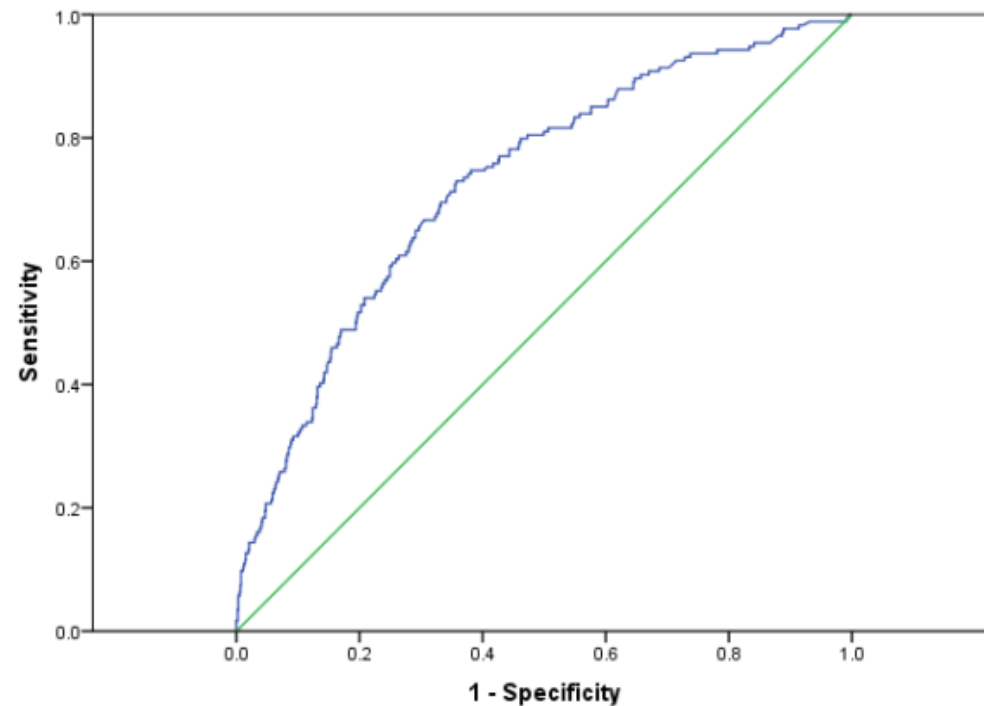
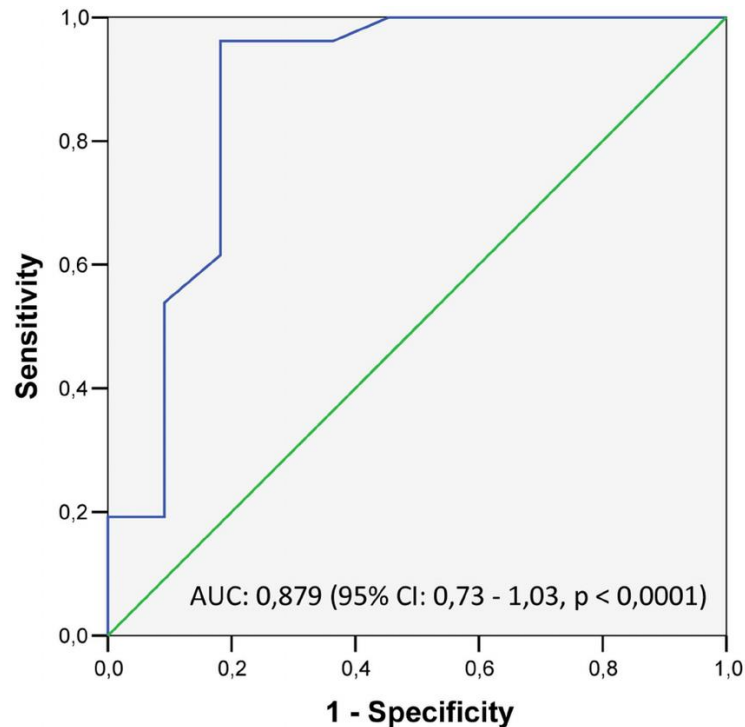
**Table does not correspond to figure*



TRUTH	LOGISTIC Regression score
1	0.98
1	0.93
0	0.88
1	0.8429
0	0.81
1	0.77
0	0.7527
0	0.7194
1	0.59
0	0.42
0	0.0839

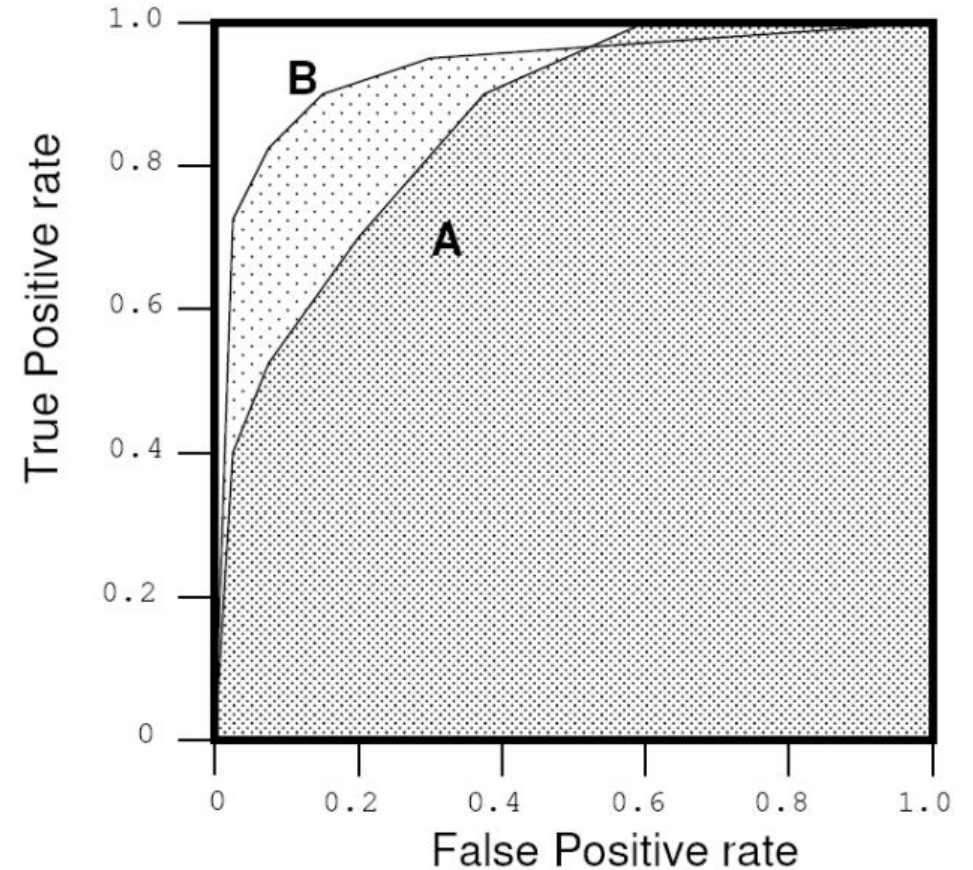
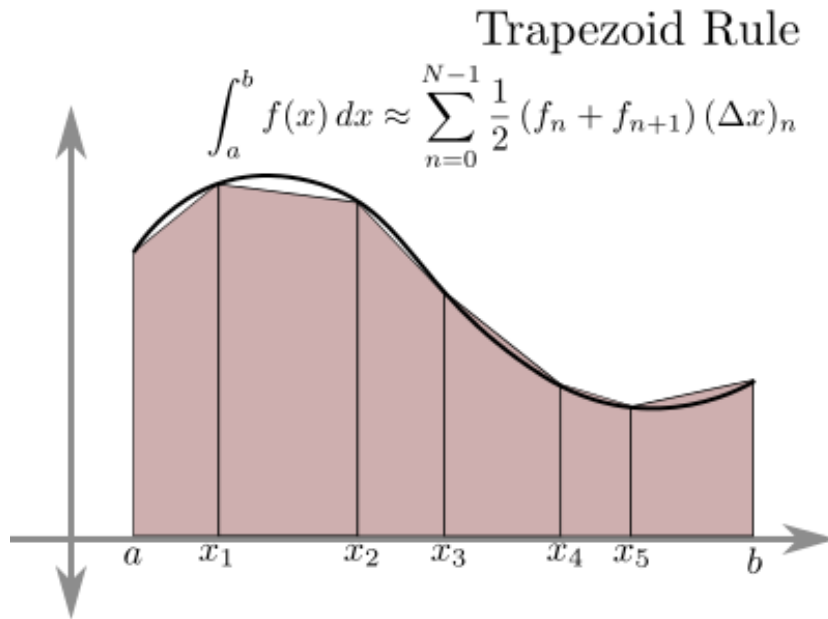
An algorithm for making an ROC curve

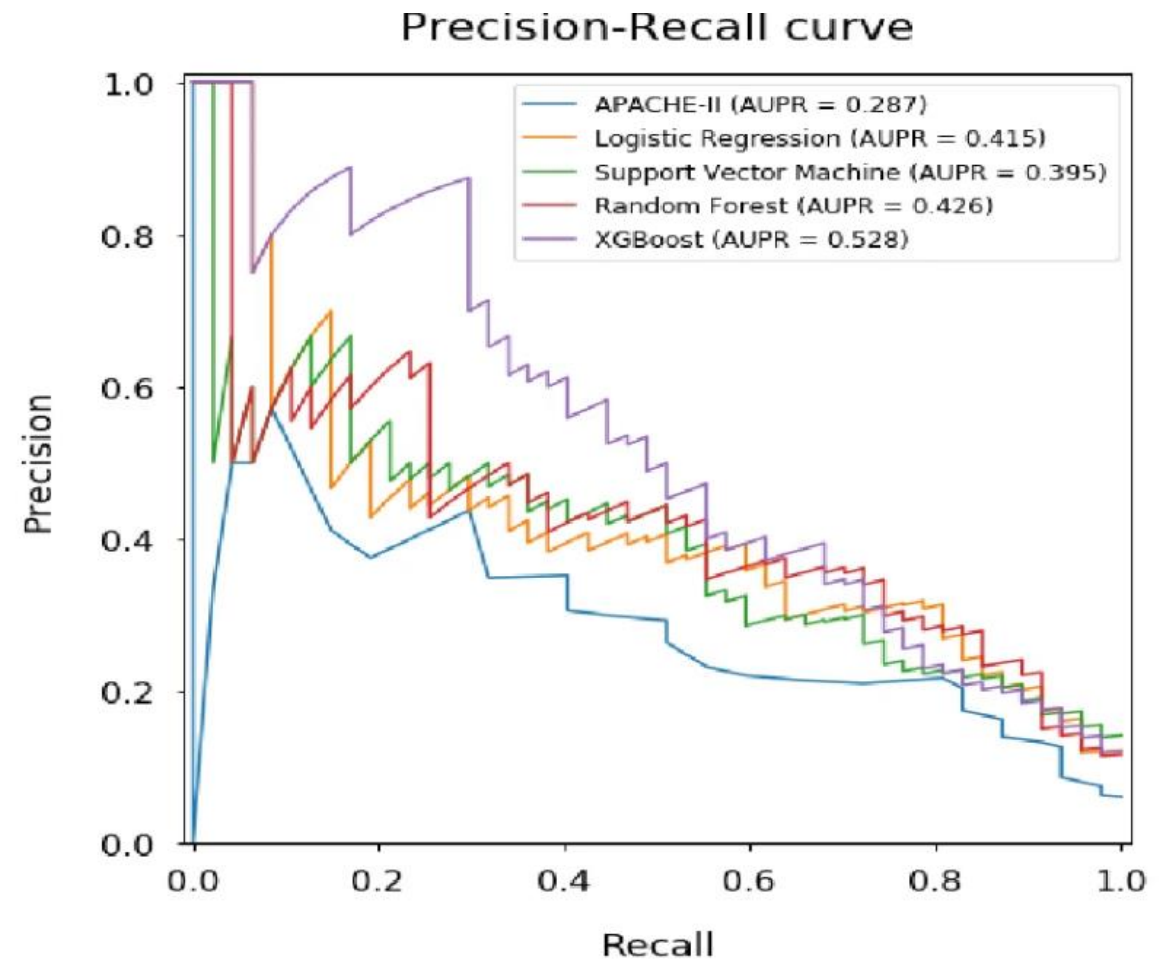
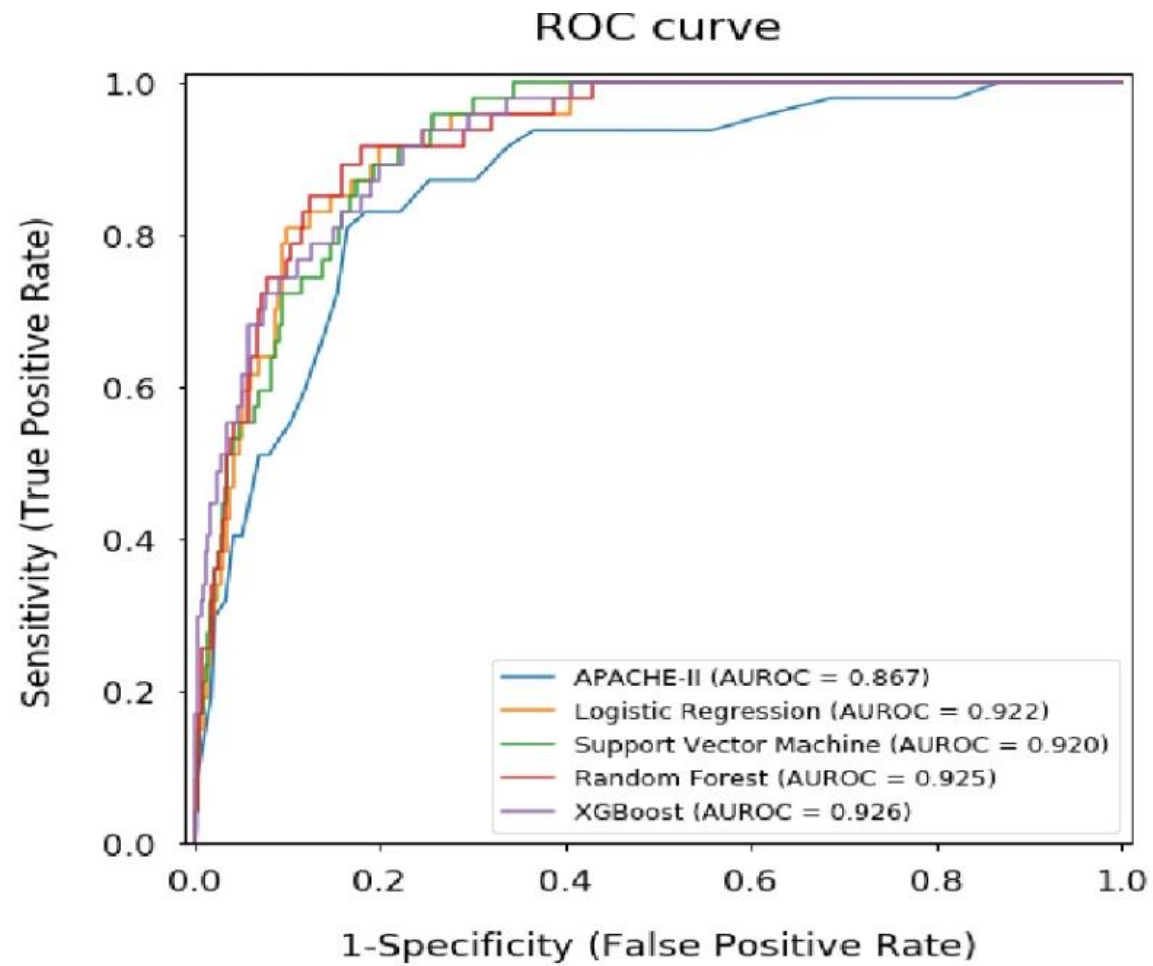
- Smoothness of the ROC curve is dependent on the # of points in it
- Restricted by # of test points and uniqueness of scores:



Piggyback to get Area Under the Curve (AUC)

Slight modification to the algorithm for constructing ROC curves—basically a simple form of integration to compute the area under the curve.





Hirano, Y., Kondo, Y., Hifumi, T. *et al.* Machine learning-based mortality prediction model for heat-related illness. *Sci Rep* **11**, 9501 (2021).

If we care about these metrics, why not use them as a loss function to train the model?

- These losses are often not differentiable everywhere (e.g. AUC and others have hard thresholding).
- Some niche attempts to make metrics differentiable, but in practice, rarely used.
- Can't use mini-batch (ranking requires entire of data), hence can't use SGD.

Learning cost-sensitive classifiers

- In regular classification (all we have seen so far), we treat all types of misclassification errors the same (e.g. in using a likelihood loss/cost function).
- A more general setting is to learn *cost-sensitive* classifiers where we allow for different types of errors to have more or less impact.

Model	Patient	
	Loss Function	
	Has Cancer	Doesn't Have Cancer
	Predicts Cancer	Low
	Predicts No Cancer	OMG Panic!
		Low

Learning cost-sensitive classifiers

- In regular classification (all we have seen so far), we treat all types of misclassification errors the same (e.g. in using a likelihood loss/cost function).
- A more general setting is to learn *cost-sensitive* classifiers where we allow for different types of errors to have more or less impact:

		Patient	
Model	<i>cost matrix</i>	Has Cancer	Doesn't Have Cancer
	Predicts Cancer	Low	Medium
	Predicts No Cancer	OMG Panic!	Low

Optimizing for Cost-Sensitive Loss

e.g., use a "Cost Balancing" loss:

$$\operatorname{argmin}_{w,b} \left(1000 \sum_{i:y_i=1} L(y_i, f(x_i | w, b)) + \sum_{i:y_i=-1} L(y_i, f(x_i | w, b)) \right)$$

user-defined
cost matrix

<i>cost matrix</i>	Has Cancer	Doesn't Have Cancer
Predicts Cancer	0	1
Predicts No Cancer	1000	0

changes the objective function during training, unlike ROC type analysis

[table from Yisong Yue]